Building Reproducible Analytical Pipelines

Master of Data Science, University of Luxembourg - 2026

Bruno Rodrigues

2026-09-15

Table of contents

Introduction **1**
Schedule 1
Reproducible analytical pipelines? 2
Why T? Focusing on what truly matters 3
The Pedagogical Justification 5
Data products? 6
Machine learning? 6
What actually is reproducibility? 7
The requirements of a RAP 7
The Reproducibility Spectrum 7
Large Language Models & AI Coding Agents 8
Setting Up an AI Agent 8
Prerequisites 10
Course Project & Grading 10
Project Scope & Ambition 10
Grading Rubric (20 Points) 12
Evaluation Protocol 15
Jargon 16
T 16
Nix 17
R & Python 17
Further Reading 17
License 18

1 Reproducible Environments with Nix **19**
1.1 Learning Outcomes 19

1.2 Why Reproducibility? Why Nix? 20
1.2.1 Problems with Ad-Hoc Tools 21
1.2.2 Nix: A Declarative Solution 21
1.3 Important Concepts 22
1.3.1 The Derivation 22
1.3.2 The Nix Store and Hash Prefixes 24
1.3.3 Component Closures 24
1.3.4 The Hermetic Sandbox 24
1.3.5 Binary Caches and Cachix 25
1.4 Setting Up Your Environment 25
1.4.1 1. Installing Nix 25
1.4.2 2. Configuring Binary Caches 26
1.4.3 3. Setting Up Positron and direnv 26
1.5 Summary 27

2 Building Pipelines with T 29
2.1 Learning Outcomes 29
2.2 Introduction: Beyond Environment Reproducibility 30
2.3 A First Look at a T Pipeline 31
2.4 Anatomy of a T Pipeline 33
2.4.1 1. The `pipeline { ... }` Block 33
2.4.2 2. Nodes: Pure Computations 33
2.4.3 3. Automatic Dependency Resolution . . . 34
2.4.4 4. Zero-Glue Serialization with Apache Arrow (`^ipc`) 34
2.4.5 5. Executing and Inspecting Pipelines . . 35
2.5 Using External Scripts and Functions 36
2.6 Summary 37

3 Pipelines as Function Composition 39
3.1 Learning Outcomes 39
3.2 Introduction: The Hidden Problem with State . . 40
3.3 Pure Functions: The Mathematical Ideal 42
3.4 T Nodes Are Pure Functions 43

3.5 A Pipeline Is Function Composition 44
3.6 Writing Clean, Pure Helper Functions 46
3.6.1 Decoupling Logic: External Scripts 46
3.7 Summary 47

4 Professional Workflows: Testing and Git 49
4.1 Introduction 50
4.2 Part 1: Unit Testing 50
4.2.1 The Philosophy of a Good Unit Test . . . 51
4.2.2 Unit Testing in Practice 52
4.2.3 Testing as a Design Tool 57
4.2.4 The Modern Data Scientist's Role: Reviewer and AI Collaborator 59
4.3 Part 2: Advanced Version Control 63
4.3.1 A Better Way to Collaborate: Trunk-Based Development 63
4.3.2 Collaborative Hygiene: Rebase vs. Merge . 65
4.3.3 Working with LLMs and Git: Managing AI-Generated Changes 66
4.4 Part 3: Testing T Pipelines with Testcraft 68
4.4.1 The `t test` Command 68
4.4.2 Agent- and CI-friendly output 68
4.4.3 The `expect_*` Primitives 71
4.4.4 Testing Node Outputs 72
4.4.5 Testing the Pipeline Structure 74
4.4.6 The Fixture-Chain Pattern 75
4.4.7 Summary of Testing Levels 76
4.5 Summary 77

5 The AI Harness: AGENTS.md, Code Inspection, and Verification 79
5.1 Learning Outcomes 80
5.2 Introduction: The Inverted Pyramid of Programming 80

5.3 Part 1: Spotting AI Mistakes: The Six Language Semantic Traps 82
5.3.1 Trap 1: Reference Semantics and In-Place Mutation 83
5.3.2 Trap 2: The "Notebook Hangover": Global Scope Capture 85
5.3.3 Trap 3: Silent Type Coercion, Truthiness, and Falsy Edge Cases 87
5.3.4 Trap 4: Floating-Point Comparisons & Numerical Tolerance 89
5.3.5 Trap 5: The "Helpful" LLM Syndrome: Silent Error Swallowing 91
5.3.6 Trap 6: API Hallucination and Deprecation Drift 92
5.4 Part 2: Building the Harness: The Role and Structure of `AGENTS.md` 93
5.4.1 What is `AGENTS.md`? 94
5.4.2 The Strategy: Translating Inspection Traps into Negative Constraints 95
5.5 An Annotated, Complete `AGENTS.md` Template . . 96
5.5.1 Exercise 2: Authoring an `AGENTS.md` . . . 105

6 A Short Intro to Packaging Your Code in R and Python 107
6.1 Introduction: Why Bother Packaging? 108
6.2 Part 1: Creating an R Package with `{usethis}` and `{devtools}` 109
6.2.1 Step 1: Project Setup 110
6.2.2 Step 2: Write and Document a Function . 111
6.2.3 Step 3: Add Unit Tests 112
6.2.4 Step 4: Check and Install 113
6.2.5 Step 5: Install from GitHub 114

6.3 Part 2: Creating a Minimal Python Package with uv . . . 115
6.3.1 Step 1: Project Setup with uv . . . 115
6.3.2 Step 2: Write a Function and Declare Dependencies . . . 116
6.3.3 Step 3: Add Unit Tests . . . 118
6.3.4 Step 4: Build and Install . . . 119
6.3.5 Step 5: Install from GitHub . . . 120
6.4 Conclusion: The Packaging Mindset in the Age of AI . . . 122
6.5 Integrating Custom Packages into T Pipelines . . . 124
6.6 Hands-on Exercises . . . 126
6.6.1 Exercise 1: Build Your First Python Package with Nix . . . 126
6.6.2 Exercise 2: Build and Package an R Tool with Nix . . . 128

7 Containerisation With Docker 131
7.1 Introduction . . . 131
7.1.1 Spatial vs Temporal Reproducibility . . . 132
7.1.2 The Best of Both Worlds . . . 133
7.1.3 Interactive Development vs Distribution . . . 134
7.2 Docker Essentials . . . 135
7.2.1 Installing Docker . . . 135
7.2.2 The Rocker Project and Image Registries . . . 137
7.2.3 Basic Docker Workflow . . . 138
7.3 Making Our Own Images . . . 140
7.3.1 A Minimal Dockerfile with Nix . . . 141
7.3.2 Splitting Into a Reusable Base Image . . . 142
7.3.3 Making T Available in the Image . . . 145
7.4 Publishing Images on Docker Hub . . . 146
7.4.1 Sharing Without Docker Hub . . . 147
7.5 What If You Don't Use Nix? . . . 147

7.6 Building Docker Images Directly with Nix 148
7.6.1 Why Skip the Dockerfile? 149
7.6.2 A Basic Example 149
7.6.3 Using T with dockerTools 151
7.6.4 Benefits Over Dockerfiles 152
7.6.5 Layered Images for Efficiency 153
7.6.6 When to Use This Approach 153
7.7 Dockerising a T Pipeline 154
7.7.1 Step 1: The Dockerfile 154
7.7.2 Step 2: The Pipeline 155
7.7.3 Step 3: Build and Run 156
7.7.4 Alternative: Using dockerTools for T . . . 157
7.8 Summary 159
7.9 Further Reading 159

8 Continuous Integration with GitHub Actions 161
8.1 Introduction 161
8.2 Getting your repo ready for GitHub Actions . . . 162
8.3 Nix and GitHub Actions 164
8.4 Running a T Pipeline on GitHub Actions 166
8.4.1 Caching T pipeline outputs between runs 168
8.5 Running a dockerized workflow 170
8.6 Building a Docker image and pushing it to a registry 173
8.7 A Complete T Pipeline Workflow on GitHub Actions 174
8.7.1 Caching Pipeline Outputs Between Runs . 175
8.7.2 The Easy Way: `pipeline_to_ga()` 177
8.8 GitHub Actions without Nix 177
8.9 Advanced patterns 179
8.9.1 Caching with Cachix 179
8.9.2 Storing outputs in orphan branches 181
8.9.3 Creating workflow summaries 182
8.10 Conclusion 184

9 Conclusion: Adopting Reproducibility in the Real World **187**
9.1 Introduction 188
9.2 Start with yourself 188
9.3 The two-minute demo 189
9.4 Make it easy for others 190
9.5 Pick your battles 190
9.6 The gradual adoption path 191
9.7 Common objections and how to address them . . 192
9.8 LLMs as adoption accelerators 193
9.9 A note on literate programming 195
9.10 Final thoughts 196

Introduction

This is the 2026 edition of the course. If you're looking for earlier editions, you can find the 2024 and 2025 archives on GitHub.

What's new in 2026:

- **T at the center**: A modern, domain-specific language purpose-built for polyglot reproducible analytical pipelines (R, Python, and Julia).
- **Pipelines as pure function composition**: Reframing pipelines through functional programming and Directed Acyclic Graphs (DAGs).
- **Universal environment reproducibility**: Using Nix as the foundation for bit-for-bit identical, cross-language software closures.
- **AI-first workflow**: Leveraging Large Language Models (LLMs) to construct and review reproducible pipelines and tests.

This course is based on my book Reproducible Polyglot Data Science.

Schedule

- **2026-10-30 (1.5 hours)**: Intro (Why T, the reproducibility spectrum) and Nix (environment reproducibility).

- **2026-11-11 (14:00 - 17:15, MSA 2.400)**: Building Pipelines with T (syntax, nodes, Arrow IPC serialization, DAG caching).
- **2026-11-19 (14:00 - 17:15, MSA 4.050)**: Pipelines as Function Composition (pure functions, immutability, state elimination).
- **2026-11-20 (10:15 - 11:45)**: Professional Workflows I: Testing and Test Oracles (unit testing with `{testthat}` & `pytest`, golden datasets, pipeline assertions with `testcraft`).
- **2026-11-27 (14:00 - 17:15, MSA 3.190)**: Professional Workflows II: The AI Harness & Collaborative Git (code inspection literacy for LLM traps, authoring `AGENTS.md`, closed-loop agent execution with `t test --json`, interactive Git with `git add -p`).
- **2026-12-03 (14:00 - 17:15, MNO 1.030)**: Packaging Code in R and Python for T pipelines.
- **2026-12-10 (14:00 - 17:15, MSA 3.120)**: Containerisation with Docker (spatial vs. temporal reproducibility, packaging Nix & T).
- **2026-12-16 (12:45 - 15:45, MSA 3.100)**: Continuous Integration with GitHub Actions & Course Project Wrap-up.

Reproducible analytical pipelines?

This course is my take on setting up code that results in some *data product*. This code has to be reproducible, documented, and production-ready. The term was originally introduced by the UK's Analysis Function.

The basic idea of a reproducible analytical pipeline (RAP) is to

have code that always produces the exact same result when run, whatever that result might be. This is crucial in research and science, but it is equally essential in industry and data-driven decision-making.

A well-documented RAP eliminates technical debt, prevents "it worked on my machine" disasters, and is readily reusable across future projects.

Why T? Focusing on what truly matters

If you have ever tried building a data science pipeline that combines both Python and R, you know the pain:

- In Python, you battle dependency hell across `pip`, `virtualenv`, `conda`, `poetry`, and `uv`, while wrestling with `PYTHONPATH` resolution and platform-specific wheels.
- In R, you configure `{renv}`, ensure compiler toolchains exist to compile C/C++/Fortran packages, and manage package library paths.
- To connect them, you end up writing fragile bash scripts or Makefiles, exporting ad-hoc CSV files that lose data types, and praying that everyone runs the scripts in the right order.

Students and practitioners end up wasting 80% of their time dealing with language-specific idiosyncrasies rather than learning reproducible architecture.

To break this cycle, this course puts **T** (`tlang`) at the center.

```
p = pipeline {
  -- 1. Load data
  data = read_csv("datasets/country_level_data.csv")

  -- 2. Clean and derive in Python
  derived = pyn(
    command = <{
      derived = data.assign(
        pct_change =
 ↪  data["average_price_m2_nominal_euros"].pct_change()
 ↪  * 100
      )
    }>,
    deserializer = ^ipc,
    serializer = ^ipc
  )

  -- 3. Visualise in R
  plot = rn(
    command = <{
      library(ggplot2)
      plot <- ggplot(derived, aes(x = year, y =
 ↪  pct_change)) +
        geom_line() +
        theme_minimal()
    }>,
    deserializer = ^ipc
  )
}

build_pipeline(p)
```

The Pedagogical Justification

T is an experimental domain-specific language that I began designing in 2025. While it is experimental, placing it at the center of this course serves a vital pedagogical purpose:

1. **Focus on Reproducibility, Not Language Quirks**: T isolates the essential principles of software reproducibility:
    - **Pure Functions**: Each node is an isolated function with explicit inputs and a single output.
    - **Directed Acyclic Graphs (DAGs)**: Execution order is derived mathematically from data dependencies, not the arbitrary layout of script files or notebook cells.
    - **Hermetic Execution**: Execution takes place inside an isolated sandbox guaranteed by Nix.
    - **Content-Addressed Caching**: Only nodes whose inputs or code have changed get rebuilt.
    - **Zero-Glue Polyglot Data Exchange**: Cross-language data transfer uses standardized formats like Apache Arrow IPC (`^ipc`), passing DataFrames between Python, R, and Julia without custom serialization code.
2. **Eliminating Fragile Glue**: You no longer need to write custom glue scripts, parse command-line arguments, or track intermediate file names.
3. **Designed for the LLM Era**: T was created with a minimalist, constraint-based syntax that LLMs can generate and reason about with high precision. You can let an LLM help write the pipeline definitions, while you and the LLM focus on the R or Python data analysis code inside each node.

By abstracting execution into T and environment management into Nix, we remove the accidental complexity of modern data science so you can master the fundamental concepts.

Data products?

In this course, each of you will develop a *data product*. A data product is anything that requires data as an input and produces value: an analytical report, a reproducible paper, an interactive dashboard, or an API.

The focus of the course is not on the end product itself (which you can choose based on your interests), but on the **pipeline** that builds that product reliably, reproducibly, and non-interactively.

Machine learning?

Being skilled in machine learning is not enough to become an effective data scientist. In fact, the model is often the easiest piece to replace. What is non-negotiable is knowing:

- How to write, test, and modularize code;
- How to manage environments deterministically;
- How to work within the command line and version control with Git;
- How to construct automated DAG pipelines;
- How to package code and deploy with Docker and Continuous Integration.

What actually is reproducibility?

A project is reproducible if it can be rerun by anyone, anywhere, at any point in time, yielding the exact same results with minimal friction.

The requirements of a RAP

For a project to qualify as a RAP, it must meet several strict criteria: - **Available, version-controlled source code** (tracked with Git and GitHub); - **Deterministic dependencies** (managed completely with Nix); - **Non-interactive execution** (run with a single command, without manual steps in GUIs or notebooks); - **Tested logic** (unit tests validating functions and pipeline data contracts); - **Open-source foundation** (reproducible operating system closures via Nix and Docker).

The Reproducibility Spectrum

Reproducibility is not binary; it exists along a continuum (Peng 2011):

A project can control for: 1. Code only; 2. Code + language versions; 3. Code + language + package libraries; 4. Code + packages + system-level OS dependencies (Nix / Docker); 5. Hardware architecture and bit-for-bit hermetic closures.

In this course, we aim for the highest practical tier of this spectrum.

Large Language Models & AI Coding Agents

LLMs and agentic coding workflows have transformed how software is developed. In this course, we treat LLMs as power tools: they are highly encouraged for drafting functions, generating tests, and structuring pipelines. However, using an LLM requires you to become a rigorous code reviewer. You will learn how to use Git's interactive staging (`git add -p`) and automated test suites to verify that AI-generated code meets your exact expectations before committing it.

Setting Up an AI Agent

It is important that you have an AI coding agent installed on your computer system for this course.

When setting up your environment, consider the following options:

1. **University-provided tools**: First, check if your university provides access to an AI coding assistant or institutional accounts (for example, through GitHub Education / Student Developer Pack for GitHub Copilot, or campus AI services).
2. **Personal accounts**: If you already have a personal account or subscription to a coding agent or AI model provider (such as Claude Code, Cursor, GitHub Copilot, or API keys for models like Claude, GPT, or Gemini), you are welcome to use it.
3. **Freebuff**: If you do not have an institutional license or personal subscription, you can use Freebuff. Freebuff provides

free, ad-supported access to DeepSeek 4.1 for 7 hours a day, which is a very good deal for completing your coursework.

4. **OpenCode**: Another option is OpenCode, which also provides free models. Keep in mind that free models on OpenCode typically come with smaller context windows, and sessions might be shorter.

💡 Model Strategy: Strong Models for Planning & Review, Flash Models for Coding

When using an AI coding agent, you will get much better results by adopting a **two-tier model strategy**:

- **Strong reasoning models for planning and review**: Use a frontier model (such as Claude Sonnet, OpenAI o-series/GPT-4o, DeepSeek Reasoner, or Gemini Pro) when initially planning your pipeline architecture, structuring your `AGENTS.md`, drafting complex DAG nodes, or reviewing tricky test failures and edge cases.
- **Fast, "flash-like" models for coding and execution**: Once the plan and interface contracts are clear, switch to a quicker, lightweight model (such as Gemini Flash, Claude Haiku, or DeepSeek Flash) to generate the boilerplate code, run iterative test loops, and apply small fixes.

This split significantly reduces latency, saves daily quota, and yields cleaner results by preventing "context bloat" during fast iterative loops while reserving deep reasoning for architectural decisions.

Prerequisites

- Basic programming familiarity with R or Python (loops, functions, data structures).
- Familiarity with the command line interface (CLI).
- Linux or macOS operating system (Windows users **must** use WSL2).
- An AI coding agent installed on your computer system (see above).

Course Project & Grading

Grading is based on your final course project. During lecture hours you will learn each concept through live demonstrations and hands-on exercises. In parallel, you will develop your own reproducible analytical pipeline around a substantial analytical problem and dataset of your choice.

Because you are actively encouraged to use AI coding agents to accelerate your development, the goal of this project is not merely to write boilerplate code. Instead, the ambition is to tackle a **challenging, multi-language analytical problem** and demonstrate your skills as a **harness architect, code reviewer, and reproducibility engineer**.

Project Scope & Ambition

To achieve a high grade, your project must meet the following specifications:

- **Polyglot Mandate**: The pipeline must integrate **at least two languages** (e.g., Python + R, or Python + Julia) orchestrated with T and exchanging data via Apache Arrow IPC (`^ipc`). Each language must be chosen deliberately for its strengths (e.g., Python for data scraping or machine learning embeddings, R for specialized statistical modeling and `{ggplot2}` visualization, or Julia for high-performance simulation).
- **Non-Trivial DAG Architecture**: The pipeline cannot be a single linear sequence of scripts. It must feature a branched Directed Acyclic Graph (DAG) with at least 5 distinct computation nodes (e.g., ingesting multiple data sources, parallel feature transformations, model fitting, and validation).
- **Pure Functions & Immutability**: All node computations must be pure functions with explicit inputs and outputs, free of global scope leakage, in-place data mutations, or unmanaged side effects.
- **The AI Harness & Test Oracles**: The repository must include a customized `AGENTS.md` specifying your project's negative constraints, paired with an automated test suite (`pytest`, `{testthat}`, or `testcraft`) anchored by golden reference datasets. Tests must verify analytical invariants and prevent LLM-generated bugs.
- **Hermetic Environment**: All dependencies (R, Python, Julia, system libraries, and Quarto) must be declared and pinned via a Nix flake (`flake.nix` and `flake.lock`).
- **Continuous Integration**: The repository must contain a GitHub Actions workflow that executes your test suite and builds the pipeline in a clean Nix environment on every push.
- **Final Data Product**: The pipeline must produce an automated, polished end product—such as a rendered Quarto

report, an interactive dashboard, or an exported analytical artifact.

Grading Rubric (20 Points)

At the end of the semester, your repository will be evaluated across four dimensions (5 points each):

Dimension	Points	Criteria & Evaluation
1. Polyglot Architecture & T Pipeline	**5**	Pipeline orchestrates ≥ 2 languages via T with Apache Arrow IPC (`~ipc`). Computation nodes are pure functions without side effects or in-place mutations. DAG caching works correctly (unchanged nodes are skipped on rebuilds).

Dimension	Points	Criteria & Evaluation
2. The AI Harness & Test Oracles	**5**	Includes a project-specific `AGENTS.md` with explicit negative constraints. Test suite (`t test --json`, `pytest`, or `testthat`) includes rigorous test oracles / golden datasets. Code is clean and free of AI semantic traps (no silent error suppression, no float `==`, no leaky globals).

Dimension	Points	Criteria & Evaluation
3. Deterministic Environment & CI	**5**	Dependencies are declared in `flake.nix` and pinned in `flake.lock`. A GitHub Actions workflow passes cleanly on every push, building and testing the project non-interactively without manual intervention.

Dimension	Points	Criteria & Evaluation
4. Problem Complexity & Data Product	**5**	Tackles a non-trivial, interesting analytical question. Produces a polished final data product (e.g., Quarto document). Repository has an exemplary `README.md` explaining the DAG architecture, data flow, and instructions to reproduce.

Evaluation Protocol

Your repository will be evaluated objectively and non-interactively:

1. **The "One-Command" Test**: The instructor clones your repository and runs the Nix build command (or verifies the green status of your GitHub Actions CI). If the pipeline fails to build from scratch or requires imperative manual steps, marks are deducted immediately.

2. **Harness & Oracle Inspection**: The instructor reviews your `AGENTS.md` and test suite to verify that your test assertions are genuine analytical oracles rather than trivial tautologies.
3. **Diff & Code Quality Review**: The instructor reviews the code inside your pipeline nodes to verify that functions are pure, properly scoped, and that AI-generated artifacts have been rigorously audited.

Jargon

T

- **T (`tlang`)**: A domain-specific language for defining reproducible analytical pipelines across R, Python, and Julia.
- **Node**: A single computation step in a pipeline. Nodes act as pure functions: they take explicit inputs and produce a single serialized artifact.
- **`rn()` / `pyn()` / `jln()`**: T node constructors for R, Python, and Julia runtimes.
- **Serializer / Deserializer**: The mechanism T uses to write and read node outputs (e.g., `^ipc` for Apache Arrow, `^csv`, `^parquet`, `^rds`).
- **Pipeline (`pipeline { ... }`)**: A declarative block that defines the DAG of nodes and their data dependencies.
- **`build_pipeline()`**: The function that compiles the DAG, checks the cache, and executes stale nodes in dependency order.
- **`read_node()`**: A function used to inspect the cached output of a specific node interactively.

Nix

- **Nixpkgs**: The central repository of over 120,000 packages used by Nix.
- **Derivation**: A low-level, cryptographic build instruction in Nix.
- **Nix Store (`/nix/store`)**: The read-only directory where all isolated packages and build artifacts reside.
- **Hermetic Sandbox**: An isolated build environment with no network access or hidden system dependencies.
- **Binary Cache**: Pre-built packages identified by cryptographic hash (e.g. Cachix or `cache.nixos.org`), avoiding redundant local compilation.

R & Python

- **CRAN / PyPI**: The respective package registries for R and Python.
- **Arrow IPC (`^ipc`)**: An in-memory columnar data format that allows zero-overhead data transfer between R and Python.
- **`testthat` / `pytest`**: Standard testing frameworks for R and Python code.

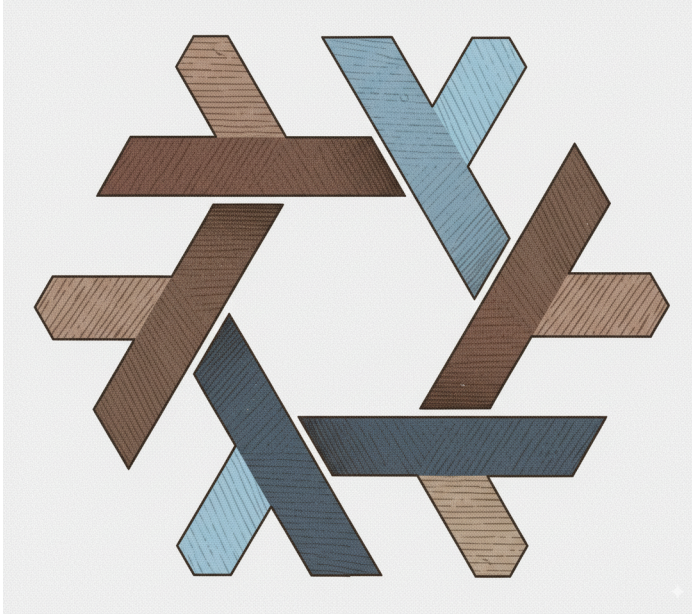
Further Reading

- Reproducible Polyglot Data Science
- T Language Repository
- Building Reproducible Analytical Pipelines with R
- How Nix Works

License

This course is licensed under the WTFPL.

1 Reproducible Environments with Nix



1.1 Learning Outcomes

By the end of this chapter, you will:

- Understand why traditional environment managers (`venv`, `renv`, `conda`) fail to achieve complete reproducibility.
- Understand core Nix concepts: derivations, closures, the immutable Nix store, and hermetic builds.
- Understand the role of binary caches (like `rstats-on-nix` and Cachix) in accelerating builds.
- Install Nix, configure binary caches, and set up Positron and `direnv` for seamless project workflows.

1.2 Why Reproducibility? Why Nix?

To ensure that an analytical project is reproducible, you must control at least four distinct layers:

1. The exact version of the programming language interpreter (Python 3.13.2, R 4.4.1, etc.).
2. The exact versions of all language-specific libraries (`pandas`, `dplyr`, `scipy`, `ggplot2`).
3. The exact versions of underlying system-level dependencies (C/C++ compilers, `libxml2`, `gdal`, `openssl`, BLAS/LAPACK, Java).
4. The underlying hardware and operating system architecture.

In everyday practice, data scientists only manage the first or second point. When someone shares an analysis, you are typically told: "Just install Python 3.11 and `pip install -r requirements.txt`." Then you run it, and it immediately crashes because a C library is missing, or a transitive dependency was updated yesterday and broke backwards compatibility.

1.2.1 Problems with Ad-Hoc Tools

Language-specific package managers solve only isolated pieces of the puzzle:

- **Python's `venv` and `uv`**: Manage Python wheels, but assume the host operating system provides all required dynamic shared libraries (`.so` or `.dylib` files). If your teammate is running Ubuntu 24.04 and you are on macOS or Arch Linux, different system library versions will lead to divergent behavior or build failures.
- **R's `{renv}`**: Records CRAN package versions and dates in a lockfile, but relies on the host system compilers and libraries to compile packages from source.
- **Docker**: Solves spatial distribution by packaging an entire Linux userland, but traditional `Dockerfile` builds are imperative (`apt-get update && apt-get install -y ...`). Rebuilding the exact same `Dockerfile` six months later will pull updated packages, introducing **temporal drift** (Malka, Zacchiroli, and Zimmermann 2024).

Traditional approaches fail to capture the complete dependency graph deterministically. This leads to the infamous "it works on my machine" syndrome.

1.2.2 Nix: A Declarative Solution

Nix takes a fundamentally different approach based on functional programming:

1. **Massive Repository (`nixpkgs`)**: With over 120,000 packages, `nixpkgs` is currently the largest open-source software

repository. It contains interpreters, development tools, system libraries, the entirety of CRAN and Bioconductor, and vast portions of PyPI.
2. **Hermetic Isolation**: Every package built by Nix is installed into its own isolated directory inside `/nix/store/`. A package never references global directories like `/usr/lib` or `/usr/local/bin`.
3. **Cryptographic Pinning**: Every package's path in the Nix store includes a cryptographic hash of all its inputs (its source code, compiler flags, and every dependency down to the C standard library). If anything in the dependency chain changes, the hash changes.
4. **Deterministic Declarations**: Instead of running imperative installation commands, you declare your desired environment in a plain-text configuration file. Anyone using that definition gets a bit-for-bit identical environment (Dolstra 2006).

1.3 Important Concepts

Before using Nix, let us establish the core conceptual foundations.

1.3.1 The Derivation

In Nix terminology, a **derivation** is:

> *A specification for running an executable on precisely defined input files to repeatably produce output files at uniquely determined file system paths.*

Think of a derivation as a pure mathematical function:

$$f(\text{inputs}, \text{build steps}) \longrightarrow \text{output}$$

If the inputs (source files, compiler, dependencies, configuration flags) are identical, the output will be bit-for-bit identical.

Here is a simple derivation written in the Nix expression language:

```
let
  pkgs = import (
    fetchTarball
     ↪  "https://github.com/rstats-on-nix/nixpkgs/archive/202
  ) {};
in
pkgs.stdenv.mkDerivation {
  name = "filtered_mtcars";
  buildInputs = [ pkgs.gawk ];
  dontUnpack = true;
  src = ./mtcars.csv;
  installPhase = ''
    mkdir -p $out
    awk -F',' 'NR==1 || $9=="1" { print }' $src >
 ↪  $out/filtered.csv
  '';
}
```

This snippet uses `awk` to filter a CSV file. Notice that `gawk` is explicitly listed as a dependency, and the output is written to a designated `$out` directory.

In this course, we will not write raw Nix derivations by hand. Instead, **T** (`tlang`) will automatically generate and evaluate these derivations for us behind the scenes!

1.3.2 The Nix Store and Hash Prefixes

All packages and derivations live under `/nix/store/`:

```
/nix/store/0agfca9pi5c93kqdac45jl9mxrwdgysx-r-4.4.1/
/nix/store/3x7y...-python3-3.13.2/
/nix/store/9k2p...-arrow-18.1.0/
```

The 32-character hash prefix is computed from the package's complete dependency graph. Because paths include this hash, multiple versions of the same library (e.g., Python 3.11, Python 3.12, and Python 3.13) can coexist on the same machine without conflict.

1.3.3 Component Closures

The **closure** of a package is the complete set of all packages, libraries, and files required to run it. If you copy a package and its entire closure to another machine running Nix, it is guaranteed to execute without needing any other software installed on that host.

1.3.4 The Hermetic Sandbox

When Nix builds a derivation, it does so in a **hermetic sandbox**:
- Network access is disabled (except for derivations explicitly declared with a fixed cryptographic hash, such as downloading

a source archive). - Environment variables are wiped clean; only explicitly declared variables exist. - System directories like `/usr/include` or `/opt` are invisible.

This guarantees that a build cannot secretly depend on a tool or file that happens to exist on your machine.

1.3.5 Binary Caches and Cachix

Because derivations are deterministic and identified by cryptographic hashes, Nix does not need to recompile software from source if someone has already built that exact hash. Nix queries **binary caches** (such as `cache.nixos.org` and `rstats-on-nix.cachix.org`) and downloads pre-built binaries directly into your store.

1.4 Setting Up Your Environment

Let us set up Nix and configure your workstation for the course.

1.4.1 1. Installing Nix

We strongly recommend the Determinate Systems Nix Installer, which works seamlessly across Linux, macOS, and Windows (via WSL2):

```
curl --proto '=https' --tlsv1.2 -sSf -L
 ↪  https://install.determinate.systems/nix | sh -s
 ↪  -- install
```

Follow the prompts on screen, then restart your terminal. Verify the installation:

```
nix --version
```

1.4.2 2. Configuring Binary Caches

To avoid compiling R packages and Python dependencies from source, configure the `rstats-on-nix` Cachix cache:

```
nix run nixpkgs#cachix use rstats-on-nix
```

This tells Nix to check `https://rstats-on-nix.cachix.org` for pre-built binaries.

1.4.3 3. Setting Up Positron and direnv

Positron is a next-generation data science IDE built on Code OSS by Posit. It has first-class support for both R and Python, with rich variable inspectors and plotting panes.

To make Positron automatically detect the R and Python interpreters managed by Nix:

1. Install `direnv`:

   ```
   nix-env -iA nixpkgs.direnv
   ```

2. Hook `direnv` into your shell (add `eval "$(direnv hook bash)"` to your `~/.bashrc` or `eval "$(direnv hook zsh)"` to `~/.zshrc`).

3. In any project root containing a `default.nix` or `flake.nix`, create a `.envrc` file containing:

   ```
   use flake
   ```

4. Run `direnv allow`.
5. Install the **direnv** extension in Positron. Now, whenever you open a project, Positron immediately activates the exact Python and R versions defined by your Nix configuration!

1.5 Summary

Nix solves the environment reproducibility problem at its root by treating software builds as pure, declarative derivations. In the next chapter, we will see how **T** leverages Nix to build polyglot pipelines without writing boilerplate glue code.

2 Building Pipelines with T



2.1 Learning Outcomes

By the end of this chapter, you will:

- Understand the difference between environment reproducibility and computational reproducibility.

- Understand the core concepts of T: pipelines, nodes, runtimes, and serialization contracts.
- Write multi-language pipelines combining Python and R using `pyn()` and `rn()`.
- Pass structured data across languages seamlessly with Apache Arrow IPC (`^ipc`).
- Understand how T discovers data dependencies and uses content-addressed caching to skip redundant computations.

2.2 Introduction: Beyond Environment Reproducibility

In the previous chapter, we explored Nix and learned how to create bit-for-bit reproducible development environments. But here is a critical reality:

> **Reproducing the environment is necessary, but it is not sufficient.**

Imagine a colleague sends you a GitHub repository. Thanks to Nix, you drop into the exact software environment in seconds. You have R 4.4.1, Python 3.13, and all libraries installed. But when you open the repository, you find:

```
01_clean.R
02_process.py
analysis_v2.R
test_model.py
run_everything.sh
clean_data_final.csv
```

Which script runs first? Which outputs feed into which inputs? Did the author run `02_process.py` before `analysis_v2.R`, or

did they manually run snippets in an interactive session? Is `clean_data_final.csv` up to date, or was it produced three months ago from an older version of `01_clean.R`?

Reproducing the environment only solves the first half of the problem. To achieve **computational reproducibility**, the workflow itself must be formally declared in a machine-readable graph.

This is where **T** (`tlang`) comes in.

2.3 A First Look at a T Pipeline

Before diving into formal grammar, let us examine an end-to-end polyglot pipeline in T:

```
p = pipeline {
  -- 1. Ingest raw CSV data
  data = read_csv("datasets/housing_data.csv")

  -- 2. Python node: Feature engineering
  cleaned_py = pyn(
    command = <{
import pandas as pd

# Filter positive prices and calculate price per
 ↪  square meter
cleaned_py = data[data["price"] > 0].copy()
cleaned_py["price_sqm"] = cleaned_py["price"] /
 ↪  cleaned_py["surface"]
    }>,
    deserializer = ^ipc,
    serializer = ^ipc
```

```
  )

  -- 3. R node: Statistical analysis and
 ↪  visualization
  plot_r = rn(
    command = <{
library(ggplot2)
library(dplyr)

plot_r <- ggplot(cleaned_py, aes(x = surface, y =
 ↪  price)) +
  geom_point(alpha = 0.5, color = "steelblue") +
  geom_smooth(method = "lm", color = "darkred") +
  theme_minimal() +
  labs(
    title = "Housing Prices in Luxembourg",
    x = "Surface (m²)",
    y = "Price (€)"
  )
    }>,
    deserializer = ^ipc
  )
}

build_pipeline(p)
```

Notice what just happened: 1. `data` loads raw data. 2. `cleaned_py` runs in Python using pandas. It receives `data` as an Apache Arrow table via `deserializer = ^ipc`, computes new columns, and serializes its output with `serializer = ^ipc`. 3. `plot_r` runs in R using `{ggplot2}`. It receives `cleaned_py` directly from Python without you writing a single line of CSV export/import glue code! 4. `build_pipeline(p)` builds every

node in dependency order and caches the results.

2.4 Anatomy of a T Pipeline

Let us break down the core components of T.

2.4.1 1. The `pipeline { ... }` Block

A pipeline is defined using a declarative block:

```
p = pipeline {
  node_a = ...
  node_b = ...
}
```

The code inside the braces is purely declarative: it defines a Directed Acyclic Graph (DAG). Nothing runs when you assign `p`. The pipeline is a first-class object that you can inspect, pass to functions, or compile.

2.4.2 2. Nodes: Pure Computations

Each named entry in a pipeline is a **node**. A node represents a single unit of computation that produces a single artifact. T provides runtime-specific constructors:

- `pyn(...)`: Declares a Python node.
- `rn(...)`: Declares an R node.
- `jln(...)`: Declares a Julia node.

- `cmd(...)`: Declares an external shell/command-line execution.
- `node(...)`: Generic node constructor where you explicitly pass `runtime = R`, `runtime = Python`, etc.

The code to execute is passed in the `command` argument using `<{ ... }>` delimiters. T does not interpret this code; it sends it to the language runtime to execute inside a hermetic Nix sandbox.

2.4.3 3. Automatic Dependency Resolution

Notice that in our example, we never explicitly told `plot_r` that it depends on `cleaned_py`.

T automatically scans the code inside `<{ ... }>` for references to upstream node names. Because the R code references `cleaned_py`, T draws a directed edge:

$$\text{data} \longrightarrow \text{cleaned_py} \longrightarrow \text{plot_r}$$

Because dependencies are inferred from names, **the order in which you write nodes inside `pipeline { ... }` does not matter**. T topological-sorts the graph and executes nodes in their true mathematical dependency order.

2.4.4 4. Zero-Glue Serialization with Apache Arrow (`^ipc`)

Passing data between different languages has historically been one of the biggest headaches in data science. Exporting intermediate CSVs causes problems: - Numbers lose precision. - Timestamps

and dates lose timezones. - Categories and factors become raw strings. - Disk I/O is slow for large datasets.

T solves this with typed serialization symbols:

- `^ipc`: Apache Arrow IPC (Inter-Process Communication). This is an in-memory, binary columnar format shared by R, Python, and Julia. When Python writes `^ipc` and R reads `^ipc`, the data is transferred with exact type preservation and zero friction.
- `^csv`: Standard comma-separated values.
- `^parquet`: Columnar compressed storage (ideal for larger datasets).
- `^rds`: Native R object serialization.
- `^pickle`: Native Python object serialization.

2.4.5 5. Executing and Inspecting Pipelines

To execute a pipeline, call `build_pipeline(p)`:

```
build_pipeline(p)
```

T inspects each node in the DAG: 1. Has this node ever been built? 2. Have any of its upstream inputs changed? 3. Has the code inside its `command` block changed?

If the answer to all three is "no", T retrieves the existing artifact from cache in microseconds. If any input has changed, only that node and its downstream dependents are rebuilt.

To inspect the result of any node interactively, use `read_node()`:

```
result = read_node(p, "cleaned_py")
```

2.5 Using External Scripts and Functions

While embedding short snippets inside `<{ ... }>` is convenient for small steps, realistic projects organize logic into external source files. T makes this straightforward:

```
p = pipeline {
  coords = rn(
    command = <{
      coords <- clean_coordinates(raw_data)
    }>,
    functions = ["src/cleaning.R"],
    deserializer = ^ipc,
    serializer = ^ipc
  )
}
```

The `functions` argument automatically sources `src/cleaning.R` into the R session before running the node's command. Similarly, Python nodes can import modules placed in `src/`.

If your node needs external files (such as raw images, reference tables, or configuration YAMLs), specify them with `include`:

```
model = pyn(
  command = <{
    # Train model using config
  }>,
```

```
  include = ["config/model_params.yaml"]
)
```

2.6 Summary

T shifts your workflow from writing imperative, error-prone scripts to declaring a deterministic DAG of computations. In the next chapter, we will examine the theoretical foundation that makes this work: why treating pipeline nodes as **pure functions** unlocks effortless caching and rock-solid reproducibility.

3 Pipelines as Function Composition



3.1 Learning Outcomes

By the end of this chapter, you will:

- Understand the fundamental problem of mutable state in scripts and computational notebooks.
- Grasp the mathematical and practical definition of pure functions (determinism and zero side effects).
- See why **every T node is a pure function**, and why **a pipeline is function composition**.
- Write pure helper functions in R and Python that plug into pipeline nodes.
- Understand how external scripts (`script = ...`) keep domain logic orchestrator-agnostic.

3.2 Introduction: The Hidden Problem with State

In the previous chapter, we introduced T and saw how to declare polyglot pipelines with nodes and serialization contracts. Now we take a step back and ask: *why* is T designed this way? Why mandatory pipelines? Why must each node produce exactly one artifact? Why does the declaration order inside `pipeline { }` not matter?

The answer lies in a body of programming principles called **functional programming** (FP). You do not need to become a lambda calculus theorist to build reproducible pipelines. But understanding the core ideas—purity, composition, and immutability—will make you a vastly better data scientist, a better author of R and Python code, and a more effective collaborator with AI assistants.

Consider a typical data analysis script:

```r
# Step 1
data <- read.csv("sales.csv")

# Step 2
data <- data[data$revenue > 0, ]

# Step 3
data$margin <- (data$revenue - data$cost) /
 ↪  data$revenue

# Step 4
model <- lm(margin ~ region, data = data)
```

```python
# Python equivalent
import pandas as pd
data = pd.read_csv("sales.csv")
data = data[data["revenue"] > 0]
data["margin"] = (data["revenue"] - data["cost"]) /
 ↪  data["revenue"]
```

This code runs. But it possesses a hidden and dangerous property: **state**. Each step mutates the variable `data` in the global environment. The script can only be understood by reading it from top to bottom, in exact sequence.

If scripting with mutable state is a crack in the foundation of reproducibility, then computational notebooks (like Jupyter) are a gaping sinkhole. Notebooks introduce an even more insidious form of state: **cell execution order**. You can run cells out of order, re-run cell 3 after cell 7, or delete a cell whose variable persists in memory. What you see on screen no longer reflects the computational state of the kernel!

This is the root problem that functional programming solves.

3.3 Pure Functions: The Mathematical Ideal

In mathematics, a function has a beautiful property: for a given input, it always returns the exact same output. The function $f(x) = x^2$ does not depend on what you computed earlier, nor does it mutate anything outside itself.

A **pure function** in programming satisfies two conditions:

1. **Deterministic**: Given the same inputs, it always produces the same output. It depends *only* on its explicit parameters—never on global variables or hidden state.
2. **Zero Side Effects**: It does not alter anything outside its scope. It does not modify global variables, rewrite files unexpectedly, or mutate input objects in place.

Compare these two functions:

```
# Impure: secretly depends on a global variable
threshold <- 0

filter_positive <- function(df) {
  df[df$value > threshold, ]  # What is threshold?
 ↪  Who defined it? When?
}

# Pure: all inputs are explicit
filter_above <- function(df, threshold) {
  df[df$value > threshold, ]  # Everything required
 ↪  is explicitly passed
}
```

The pure version is predictable. You can test it in isolation, run it in parallel, and reason about it without needing to know what happened elsewhere in the program.

3.4 T Nodes Are Pure Functions

This is not a metaphor. A T pipeline node is, by construction, a pure function:

1. **Explicit Inputs**: Upstream data is passed into the node's sandbox as named inputs. There is no global environment to accidentally read from.
2. **Explicit Single Output**: A node returns exactly one thing: the artifact written to the Nix store by its serializer. A node cannot secretly alter another node's output.
3. **Hermetic Execution**: Because Nix runs the node inside a hermetic sandbox with no outside network or filesystem leakage, the execution is guaranteed to be pure.

Look at a concrete example:

```
p = pipeline {
  -- Node 1: Produce data. Pure: inputs = none,
 ↪  output = DataFrame (^ipc)
  data_node = rn(
    command = <{
      data.frame(
        x = c(1.0, 2.0, 3.0, 4.0, 5.0),
        y = c(0.0, 0.0, 1.0, 1.0, 1.0)
      )
    }>,
    serializer = ^ipc
```

```
  )

  -- Node 2: Fit model. Pure: inputs = data_node
 ↪  (^ipc), output = model (^pmml)
  model_node = rn(
    command = <{
      data_node$y <- as.factor(data_node$y)
      glm(y ~ x, data = data_node, family =
 ↪  binomial(link = "logit"))
    }>,
    deserializer = ^ipc,
    serializer = ^pmml
  )
}

build_pipeline(p)
```

`data_node` takes no inputs and returns a DataFrame. `model_node` takes one DataFrame (`data_node`) and produces a model artifact. Neither can reach outside its sandbox.

Because nodes are pure functions, **T can cache them safely**: if `data_node` has not changed, T never re-evaluates it. Furthermore, the declaration order inside `pipeline { }` does not matter. T infers the execution graph solely from data dependencies.

3.5 A Pipeline Is Function Composition

When you write a pipeline in T, you are composing pure functions:

$$\text{Pipeline} = (f_n \circ ... \circ f_2 \circ f_1)(\text{data})$$

This is identical to the pipe operator in R (`|>`) or method chaining in pandas:

```
# Function composition in R
mtcars |>
  filter(am == 1) |>
  select(mpg)
```

T elevates this concept to the architectural level. Each step is an explicit, named, cached pure function that can run in any language.

```
-- Pure R node
raw = rn(command = <{ read.csv("data.csv") }>,
 ↪  serializer = ^ipc)

-- Pure Python node composing on R output
transformed = pyn(
  command = <{
    transformed = raw.assign(log_val =
 ↪  numpy.log(raw["val"]))
  }>,
  deserializer = ^ipc,
  serializer = ^ipc
)
```

Because both nodes are pure, the serializer/deserializer contract (`^ipc`) functions as the type signature between them. The underlying runtime is an implementation detail!

3.6 Writing Clean, Pure Helper Functions

The pure function model applies not just to T nodes as a whole, but to the R and Python code you write *inside* those nodes.

To keep your code reproducible: - **Never mutate inputs in place**: Return a new object instead of modifying an existing one. - **Never rely on global variables**: Pass all configuration parameters explicitly. - **Isolate file I/O**: Separate functions that calculate values from functions that read/write disks.

3.6.1 Decoupling Logic: External Scripts

You do not have to write all your code inline inside `<{ ... }>`. In fact, professional projects keep domain logic in plain `.R` or `.py` files using the `script` parameter:

```
p = pipeline {
  cleaned = rn(
    script = "src/clean.R",
    functions = ["src/utils.R"],
    serializer = ^ipc
  )

  model = pyn(
    script = "src/train.py",
    deserializer = ^ipc,
    serializer = ^pmml
  )
}
```

Notice the tremendous advantage of this approach: **`src/clean.R` and `src/train.py` carry zero T-specific syntax**. They are

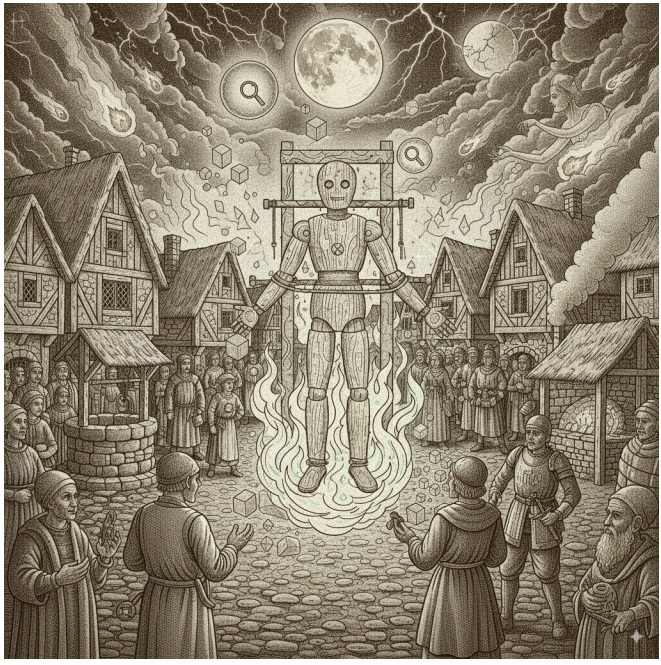
standard, idiomatic R and Python scripts. They can be: - Sourced and debugged interactively in your IDE. - Tested using standard tools (`pytest`, `testthat`). - Reused by colleagues who do not use T.

T acts purely as the orchestration layer: handling scheduling, caching, sandboxing, and data exchange.

3.7 Summary

- Mutable state in scripts and notebooks creates fragile, order-dependent code.
- Pure functions are deterministic and side-effect free.
- T nodes are pure functions running in isolated Nix sandboxes.
- A pipeline is a Directed Acyclic Graph of function compositions.
- Writing your analytical code in plain, pure R and Python files keeps your work testable and portable.

4 Professional Workflows: Testing and Git



4.1 Introduction

In the previous chapter, we learned that pure functions, those with no side effects and deterministic outputs, are the building blocks of reliable code. Small, testable functions are the foundation of reproducible pipelines.

This brings us to two final, crucial questions: how do we prove that our functions actually do what we claim they do? And how do we manage the complexity of collaborating with others (and with AI) without breaking everything?

This chapter addresses both questions by introducing unit testing and advanced Git workflows. You will learn what unit tests are and why they are essential for reliable data analysis, how to write and run them in both R (with `{testthat}`) and Python (with `pytest`), and how to use LLMs to accelerate test writing while embracing your role as a code reviewer. We will also cover professional Git techniques like Trunk-Based Development and interactive staging, which are especially valuable when managing code generated by AI assistants.

4.2 Part 1: Unit Testing

The answer to "how do we prove it works?" is unit testing. A unit test is a piece of code whose sole job is to check that another piece of code, a "unit", works correctly. In our functional world, the "unit" is almost always a single function. This is why we spent so much time on FP in the previous chapter. Small, pure functions are not just easy to reason about; they are incredibly easy to test.

Writing tests is your contract with your collaborators and your future self. It's a formal promise that your function, `calculate_mean_mpg()`, given a specific input, will always produce a specific, correct output. It's the safety net that catches bugs before they make it into your final analysis and the tool that gives you the confidence to refactor and improve your code without breaking it.

4.2.1 The Philosophy of a Good Unit Test

So, what should we test? Writing good tests is a skill, but it revolves around answering a few key questions about your function. For any function you write, you should have tests that cover:

- The "Happy Path": does the function return the expected, correct value for a typical, valid input?
- Bad Inputs: does the function fail gracefully or throw an informative error when given garbage input (e.g., a string instead of a number, a data frame with the wrong columns)?
- Edge Cases: how does the function handle tricky but valid inputs? For example, what happens if it receives an empty data frame, a vector with `NA` values, or a vector where all the numbers are the same?

Writing tests forces you to think through these scenarios, and in doing so, almost always leads you to write more robust and well-designed functions.

4.2.2 Unit Testing in Practice

Let's imagine we've written a simple helper function to normalise a numeric vector (i.e., scale it to have a mean of 0 and a standard deviation of 1). We'll save this in a file named `utils.R` or `utils.py`.

R version (`utils.R`):

```r
normalize_vector <- function(x) {
  (x - mean(x, na.rm = TRUE)) / sd(x, na.rm = TRUE)
}
```

Python version (`utils.py`):

```python
import numpy as np

def normalize_vector(x):
  return (x - np.nanmean(x)) / np.nanstd(x)
```

Now, let's write tests for it.

4.2.2.1 Testing in R with `{testthat}`

In R, the standard for unit testing is the `{testthat}` package. The convention is to create a `tests/testthat/` directory in your project, and for a script `utils.R`, you would create a test file named `test-utils.R`.

Inside `test-utils.R`, we use the `test_that()` function to group related expectations.

```
# In file: tests/testthat/test-utils.R

# First, we need to load the function we want to
 ↪  test
source("../../utils.R")

library(testthat)

test_that("Normalization works on a simple vector
 ↪  (the happy path)", {
  # 1. Setup: Create input and expected output
  input_vector <- c(10, 20, 30)
  expected_output <- c(-1, 0, 1)

  # 2. Action: Run the function
  actual_output <- normalize_vector(input_vector)

  # 3. Expectation: Check if the actual output
   ↪  matches the expected output
  expect_equal(actual_output, expected_output)
})

test_that("Normalization handles NA values
 ↪  correctly", {
  input_with_na <- c(10, 20, 30, NA)
  expected_output <- c(-1, 0, 1, NA)

  actual_output <- normalize_vector(input_with_na)

  # We need to use expect_equal because it knows how
   ↪  to compare NAs
  expect_equal(actual_output, expected_output)
})
```

The `expect_equal()` function checks for near-exact equality. `{testthat}` has many other `expect_*()` functions, like `expect_error()` to check that a function fails correctly, or `expect_warning()` to check for warnings.

4.2.2.2 Testing in Python with `pytest`

In Python, the de facto standard is `pytest`. It's incredibly simple and powerful. The convention is to create a `tests/` directory, and your test files should be named `test_*.py`. Inside, you just write functions whose names start with `test_` and use Python's standard `assert` keyword.

```
# In file: tests/test_utils.py

import numpy as np
from utils import normalize_vector # Import our
 ↪  function

def test_normalize_vector_happy_path():
    # 1. Setup
    input_vector = np.array([10, 20, 30])
    expected_output = np.array([-1.0, 0.0, 1.0])

    # 2. Action
    actual_output = normalize_vector(input_vector)

    # 3. Expectation
    # For floating point numbers, it's better to
     ↪  check for "close enough"
    assert np.allclose(actual_output,
     ↪  expected_output)
```

```
def test_normalize_vector_with_nas():
    input_with_na = np.array([10, 20, 30, np.nan])
    expected_output = np.array([-1.0, 0.0, 1.0,
 ↪  np.nan])

    actual_output = normalize_vector(input_with_na)

    # `np.allclose` doesn't handle NaNs, but
     ↪  `np.testing.assert_allclose` does!
    np.testing.assert_allclose(actual_output,
 ↪  expected_output)
```

Because this isn't a package though (yet), but a simple project with scripts, you also need to create another file called `pytest.ini`, which will tell `pytest` where to find the tests:

```
[pytest]
# Discover tests in the tests/ directory
testpaths = tests/

# Include default patterns and your hyphenated
 ↪  pattern
python_files = test_*.py *_test.py test-*.py

# Adds the root directory to the pythonpath, without
 ↪  this
# it'll be impossible to import normalize_vector()
 ↪  from
# utils.py
pythonpath = .
```

To run your tests, you simply navigate to your project's root directory in the terminal and run the command `pytest`. It will

automatically discover and run all your tests for you. That being said, you will get an error:

```
>       assert np.allclose(actual_output,
 ↪  expected_output)
E       assert False
E        +  where False = <function allclose at
 ↪  0x7f0e81c959f0>(array([-1.22474487,  0.        ,
 ↪  1.22474487]), array([-1.,  0.,  1.]))    def
 ↪  test_normalize_vector_with_nas():
        input_with_na = np.array([10, 20, 30,
 ↪  np.nan])
        expected_output = np.array([-1.0, 0.0, 1.0,
 ↪  np.nan])

        actual_output =
 ↪  normalize_vector(input_with_na)

        # `np.allclose` doesn't handle NaNs, but
         ↪  `np.testing.assert_allclose` does!
>       np.testing.assert_allclose(actual_output,
 ↪  expected_output)
E       AssertionError:
E       Not equal to tolerance rtol=1e-07, atol=0
E
E       Mismatched elements: 2 / 4 (50%)
E       Max absolute difference among violations:
 ↪  0.22474487
E       Max relative difference among violations:
 ↪  0.22474487
E        ACTUAL: array([-1.224745,  0.      ,
 ↪  1.224745,       nan])
E        DESIRED: array([-1.,  0.,  1., nan])
```

```
tests/test_utils.py:25: AssertionError
=========================================================
 ↪  short test summary info
 ↪  =========================================================
FAILED
 ↪  tests/test_utils.py::test_normalize_vector_happy_path
 ↪  - assert False
FAILED
 ↪  tests/test_utils.py::test_normalize_vector_with_nas
 ↪  - AssertionError:
=========================================================
 ↪  2 failed in 0.15s
 ↪  =========================================================
```

You don't encounter this issue in R and understanding why reveals how valuable unit tests can be. (Hint: how does the implementation of the `sd` function, which computes the standard deviation, differ between R and NumPy?)

4.2.3 Testing as a Design Tool

Testing can also help you with programming, by thinking about edge cases. For example, what happens if we try to normalise a vector where all the elements are the same?

Let's write a test for this edge case first.

`pytest` version:

```
# tests/test_utils.py
def test_normalize_vector_with_zero_std():
    input_vector = np.array([5, 5, 5, 5])
```

```
    actual_output = normalize_vector(input_vector)
    # The current function will return `[nan, nan,
     ↪  nan, nan]`
    # Let's assert that we expect a vector of zeros
     ↪  instead.
    assert np.allclose(actual_output, np.array([0,
     ↪  0, 0, 0]))
```

If we run `pytest` now, this test will fail. Our test has just revealed a flaw in our function's design. This process is a core part of Test-Driven Development (TDD): write a failing, but correct, test, then write the code to make it pass.

Let's improve our function:

```
import numpy as np

def normalize_vector(x):
  std_dev = np.nanstd(x)
  if std_dev == 0:
    # If std is 0, all elements are the mean. Return
     ↪  a vector of zeros.
    return np.zeros_like(x, dtype=float)
  return (x - np.nanmean(x)) / std_dev
```

Now, if we run `pytest` again, our new test will pass. We used testing not just to verify our code, but to actively make it more robust and thoughtful.

4.2.4 The Modern Data Scientist's Role: Reviewer and AI Collaborator

In the past, writing tests was often seen as a chore. Today, LLMs make this process very easy.

4.2.4.1 Using LLMs to Write Tests

LLMs are fantastic at writing unit tests. They are good at handling boilerplate code and *thinking* of edge cases. You can provide your function to an LLM and give it a prompt like this:

> "Here is my Python function `normalize_vector`. Please write three `pytest` unit tests for it. Include a test for the happy path with a simple array, a test for an array containing `np.nan`, and a test for the edge case where all elements in the array are identical."

The LLM will likely generate high-quality test code that is very similar to what we wrote above.



All you need is context

When using an LLM to generate tests, context is everything. If you are writing tests for functions that heavily use specific packages (like `{dplyr}` or `pandas`), providing the LLM with that package's API reference ensures it writes idiomatic and correct tests.
For example, when testing T pipeline code, point the LLM at the project's `AGENTS.md` and `T-LANGUAGE-REFERENCE.md` so it knows the exact syntax and runtime behaviour of the nodes it is testing. This is particularly useful for packages that are not very well-known (or internal packages to your

> company that aren't even public) or packages that have been updated very recently, as the LLM's training data cutoff might not include the latest versions of the package.

This is a massive productivity boost. However, this introduces a new, critical role for the data scientist: you are the reviewer.

An LLM does not write your tests; it generates a draft. It is your professional responsibility to:

1. Read and understand every line of the test code.
2. Verify that the `expected_output` is actually correct.
3. Confirm that the tests cover the cases you care about.
4. Commit that code under your name, taking full ownership of it.

"A COMPUTER CAN NEVER BE HELD ACCOUNTABLE, THEREFORE A COMPUTER MUST NEVER MAKE A MANAGEMENT DECISION" - IBM Training Manual, 1979.

This principle is, in my opinion, even more important for tests than for production code. Even before LLMs, we relied on code we didn't write ourselves. At my first job, my boss insisted that I avoid external packages entirely and rewrite everything from scratch. I ignored her and continued using the packages I trusted. At some point, you have to trust strangers. That was true then, and it is true now, except that "strangers" now includes LLMs and developers who themselves rely heavily on LLMs.

But here is the key difference: tests are small. Unlike a sprawling codebase, a unit test is short enough to read and understand completely. When you review an LLM-generated test, you can verify that the expected output is correct and that the test covers the right cases. This makes tests uniquely suited to the "trust but verify" workflow that AI-assisted development demands.

4.2.4.2 The Test Oracle Problem: Establishing Ground Truth for AI

When you prompt an AI assistant to write both your analytical code and its corresponding unit tests, you run headfirst into a subtle and dangerous trap: **the test oracle problem**.

In software engineering, a *test oracle* is the mechanism that determines whether a program's output is correct. If you ask an LLM to implement a complex statistical calculation (such as computing weighted quantiles or calculating a Gini coefficient) and then ask the same LLM to write the unit test, the model will happily write an assertion that verifies its own implementation—even if its underlying math is subtly wrong, biased, or missing an edge case. The test passes, giving you a false sense of security.

For AI-assisted data science to be safe and efficient, **the human must provide or verify the oracle**. You do not need to write every line of code, but you must prepare the ground truth before trusting the model's implementation.

In practice, there are three powerful types of oracles you should prepare:

1. **Golden Datasets (Snapshot Testing):** A golden dataset is a small, hand-crafted, or authoritatively verified table of inputs and expected outputs. You can compute the expected values by hand for a three-row table, extract them from a trusted textbook, or export them from an established legacy system. When the LLM generates a vectorized pipeline node in Python or R, your test asserts that it produces bit-for-bit or numerically identical results to the golden snapshot.

2. **Reference Implementations (Dual-Running):** Suppose you want an LLM to write a high-performance, parallelized data cleaning routine, or port an algorithm across languages (for example, from R to Python or Julia). Rather than verifying the new implementation by eye, you use the existing, trusted code as the oracle. You run both implementations on the same sample inputs and assert that their outputs match:

```
def test_optimized_against_reference():
    sample_input = generate_sample_data()
    # Reference oracle: trusted, simple, but
     ↪ perhaps slow
    expected = reference_cleaner(sample_input)
    # AI-generated candidate: fast, complex, or
     ↪ cross-language
    actual = fast_ai_cleaner(sample_input)
    pd.testing.assert_frame_equal(actual,
 ↪ expected)
```

3. **Property-Based Oracles and Invariants:** Sometimes you cannot predict the exact output for every arbitrary input, but you know the mathematical laws that *must* hold true. In property-based testing, you assert these invariants:

    - *Conservation laws:* Total revenue across subgroups must equal total unfiltered revenue.
    - *Bounded ranges:* Probabilities must always fall in $[0, 1]$; normalized Z-scores must have a sample mean of 0 and standard deviation of 1.
    - *Idempotence:* Running the cleaner twice on the same data must produce the exact same output as running it once (`clean(clean(x)) == clean(x)`).
    - *Set preservation:* Sorting or deriving fields must never

silently drop unique row IDs or invent new records.

By preparing oracles upfront, you create an objective verification harness. The LLM can draft, refactor, and optimize code freely, and you have an automated, mathematically grounded referee to verify that the results are genuinely correct.

4.3 Part 2: Advanced Version Control

I assume you already know the basics of Git: `clone`, `add`, `commit`, and `push`. If you don't, there are countless tutorials available, and I highly recommend you check one out and then come back to this.

In this section, we will focus on the techniques that separate the "I submit code via Dropbox" novice from the professional data scientist: Trunk-Based Development and managing AI-generated code.

4.3.1 A Better Way to Collaborate: Trunk-Based Development

A common mistake for new teams is to use branches to add new features, and then let these feature branches live for a very long time. A data scientist might create a branch called `feature-big-analysis`, work on it for three weeks, and then try to merge it back into `main`. The result is often what's called "merge hell": `main` has changed so much in three weeks that merging the branch back in creates dozens of conflicts and is a painful, stressful process.

To avoid this, many professional teams use a workflow called Trunk-Based Development (TBD). The philosophy is simple but powerful:

> All developers integrate their work back into the main branch (the "trunk") as frequently as possible, at least once a day.

This means that feature branches are incredibly short-lived. Instead of a single, massive feature branch that takes weeks, you create many tiny branches that each take a few hours or a day at most.

4.3.1.1 How to Work with Short-Lived Branches

But how can you merge something back into `main` if the feature isn't finished? The `main` branch must always be stable and runnable. You can't merge broken code.

The first way to solve this issue is to use feature flags.

A feature flag is just a simple variable (like a `TRUE`/`FALSE` switch) that lets you turn a new, unfinished part of the code on or off. This allows you to merge the code into `main` while keeping it "off" until it's ready.

Imagine you are adding a new, complex plot to `analysis.R`, but it will take a few days to get right.

```
# At the top of your analysis.R script
# --- Configuration ---
use_new_scatterplot <- FALSE # Set to FALSE while in
 ↪  development

# ... lots of existing, working code ...
```

```
# --- New Feature Code ---
if (use_new_scatterplot) {
  # All your new, unfinished, possibly-buggy
   ↪ plotting code goes here.
  # It won't run as long as the flag is FALSE.
  library(scatterplot3d)
  scatterplot3d(mtcars$mpg, mtcars$hp, mtcars$wt)
}
```

With this `if` block, you can safely merge your changes into `main`. The new code is there, but it won't execute and won't break the existing analysis. Other developers can pull your changes and won't even notice. Once you've finished the feature in subsequent small commits, the final change is just to flip the switch: `use_new_scatterplot <- TRUE`.

4.3.2 Collaborative Hygiene: Rebase vs. Merge

When you and a colleague both work on `main`, your histories can diverge. When you `git pull`, Git behaves in two ways:

1. Merge (Default): creates a "merge commit" that ties the histories together. This results in a messy, non-linear history graph if done frequently.
2. Rebase (Recommended): temporarily lifts your commits, pulls your colleague's changes, and then replays your commits on top.

In data science projects, where identifying when a plot or number changed is critical, a linear history is valuable. I strongly recommend configuring Git to use rebase by default:

```
git config --global pull.rebase true
```

Now, when you `git pull`, your local changes will stay at the "tip" of the history, keeping the log clean.

4.3.3 Working with LLMs and Git: Managing AI-Generated Changes

When working with LLMs like GitHub Copilot, it's crucial to review changes carefully before committing them. Git provides a powerful tool for this: interactive staging.

4.3.3.1 Interactive Staging: Accepting changes chunk by chunk

Git's interactive staging feature (`git add -p`) is perfect for reviewing LLM changes. Instead of blindly adding all changes with `git add .`, `git add -p` lets you review each "hunk" (chunk of changes) individually.

Suppose an LLM refactored your Python script. You run:

```
git add -p analysis.py
```

Git will show you a chunk of changes and prompt you:

```
@@ -1,2 +1,4 @@
 # Load required libraries
 import pandas as pd
+import matplotlib.pyplot as plt
```

```
+import seaborn as sns
Stage this hunk [y,n,q,a,d,s,e,?]?
```

You can reply:

- `y`: Yes, stage this (I approve).
- `n`: No, do not stage this (I reject or want to edit later).
- `s`: Split this hunk into smaller pieces (if the LLM did two unrelated things in one block).
- `e`: Edit the hunk manually before staging.

This forces you to be the reviewer. It prevents "accidental" AI hallucinations (like deleting a critical import) from slipping into your repository.

4.3.3.2 Example LLM Workflow

1. Save state: `git commit -m "Working state before LLM"`
2. Prompt LLM: "Please refactor this function to be more efficient."
3. Review: Run `git diff` to see what it did.
4. Select: Run `git add -p` and say `y` only to the parts that look correct.
5. Clean: Run `git checkout .` to discard the parts you rejected.
6. Verify: Run your unit tests!

This workflow ensures you maintain full control over your codebase while benefiting from LLM assistance.

4.4 Part 3: Testing T Pipelines with Testcraft

The R and Python testing we have just covered handles the code *inside* your nodes. But what about the pipeline itself? Does it have the right structure? Do nodes depend on each other in the right order? Does the data flowing between nodes have the right shape after a build?

T has a built-in testing system for this: the **testcraft** package and the `t test` CLI command.

4.4.1 The `t test` Command

When you initialise a T project with `t init`, the project gets a `tests/` directory alongside `src/`. Any `.t` file in `tests/` whose name starts with `test-` is automatically discovered and run by:

```
t test
```

This is the T equivalent of `pytest` or `testthat::test_dir()`. Each test file is a T script that builds a pipeline (or reads from an already-built one), makes assertions using `expect_*` functions, and fails loudly if any assertion does not hold.

4.4.2 Agent- and CI-friendly output

The `t test` command can emit machine-readable results, which is exactly what an AI agent or a CI script needs to decide what

to do next without parsing human-friendly text. Pass `--json` for a structured summary of the whole run:

```
t test --json tests/
```

```
{
  "schema_version": "1",
  "status": "passed",
  "total": 15,
  "passed": 14,
  "failed": 1,
  "duration_ms": 2340,
  "results": [
    { "file": "tests/test_arithmetic.t", "status":
    ↪  "passed", "duration_ms": 120, "error": null
    ↪  },
    { "file": "tests/test_strings.t", "status":
    ↪  "failed", "duration_ms": 85, "error":
    ↪  "Assertion failed at line 42: expected
    ↪  \"hello\" but got \"world\"" }
  ]
}
```

The agent loop is then straightforward: run `t test --json`, read the top-level `status`, and only if it is `"failed"`, read the `error` field of each failing result to decide the fix.

Notice why the **oracles** we discussed in Part 1 are indispensable here. An autonomous agent cannot navigate by vibes; it needs unambiguous, objective exit criteria. If the tests in your `tests/` directory are vague or self-referential, an agent might "fix" a broken test simply by modifying the assertion to match its own buggy output, or by writing a trivial stub that returns a hardcoded

value. When your test suite is anchored by strict oracles—golden datasets, invariant checks, and reference implementations—the agent has an uncompromising benchmark. It can propose an edit, execute `t test --json`, inspect the precise structural or numerical discrepancy, and iterate autonomously until the oracle is satisfied.

For CI, `--format junit` emits JUnit XML that runners and dashboards understand:

```
t test --format junit tests/
```

You can also filter what runs:

```
t test --only "stats"     # run only tests matching
 ↪  "stats"
t test --not "slow"       # skip tests matching
 ↪  "slow"
t test --failfast         # stop on the first
 ↪  failure
t test --list             # list tests without
 ↪  running them
t test --timeout 30       # mark tests slower than
 ↪  30s as failed
```

To exclude whole files or directories automatically, add a `tests/.tignore`:

```
# tests/.tignore
slow_integration.t
*_benchmark.t
legacy/
```

Tip

In the REPL. `t_test()` returns a DataFrame with the same results, so you can inspect failures interactively:

```
results = t_test()
failed = results |> filter($status == "failed")
nrow(failed)   -- 0 if everything passed
```

4.4.3 The `expect_*` Primitives

Testcraft's `expect_*` functions return a typed `Expect` value, `Expect_pass`, `Expect_stop`, or `Expect_hold`, rather than a bare boolean. This matters because when you wrap one in `assert()`, a failure gives you a rich diagnostic message instead of a generic `AssertionError: false`:

```
-- Bare assert: not very helpful on failure
assert(nrow(df) == 4)          -- AssertionError:
 ↪  expression evaluated to false

-- expect_*: tells you exactly what went wrong
assert(expect_nrow(df, 4))     -- AssertionError:
 ↪  expected 4 rows, got 7
assert(expect_colnames(df, ["id", "val"]))
  -- AssertionError: column mismatch: got ["id",
 ↪  "score"], expected ["id", "val"]
```

The core expectations you will use most often:

Function	What it checks
`expect_equal(actual, expected)`	Deep equality with tolerance for floats
`expect_nrow(df, n)`	DataFrame has exactly `n` rows
`expect_ncol(df, n)`	DataFrame has exactly `n` columns
`expect_colnames(df, names)`	Column names match exactly (order-sensitive)
`expect_has_colnames(df, names)`	DataFrame contains *at least* these columns
`expect_no_na(df, col)`	No NA values in `col`
`expect_type(x, "Int")`	Value has the expected type
`expect_error(expr)`	Expression produces an Error value
`expect_gt/gte/lt/lte(a, b)`	Numeric comparisons
`expect_between(x, lo, hi)`	Value falls within bounds
`expect_unique(x)`	All elements are distinct
`expect_match(str, pattern)`	String matches a regex

4.4.4 Testing Node Outputs

The typical pattern is: build the pipeline, read each node with `read_node()`, then assert on the result. Here is a complete test file for the GLM pipeline from the previous chapter:

```
-- tests/test-glm-pipeline.t
import stats
import dataframe
```

```
p = pipeline {
  data_node = node(
    command = <{
      data.frame(
        x = c(1.0, 2.0, 3.0, 4.0, 5.0),
        y = c(0L, 0L, 1L, 1L, 1L)
      )
    }>,
    runtime = R,
    serializer = ^ipc
  )
  model_node = node(
    command = <{
      data_node$y <- as.factor(data_node$y)
      glm(y ~ x, data = data_node, family =
 ↪  binomial(link = "logit"))
    }>,
    runtime = R,
    serializer = ^pmml,
    deserializer = ^ipc
  )
}

build_pipeline(p)

-- Test node outputs
data = read_node(p.data_node)
assert(expect_nrow(data, 5))
assert(expect_colnames(data, ["x", "y"]))
assert(expect_no_na(data, "x"))

model = read_node(p.model_node)
```

```
assert(expect_type(model, "Model"))

print("  glm pipeline: all assertions passed")
```

4.4.5 Testing the Pipeline Structure

Testcraft also provides expectations that operate directly on the pipeline DAG, before or after building. These are unique to T and have no equivalent in testthat or pytest: they test the *structure* of the computation, not just the outputs.

```
-- Check that p is a valid Pipeline value
assert(expect_pipeline(p))

-- Check that the pipeline contains exactly these
 ↪  nodes
assert(expect_nodes(p, ["data_node", "model_node"]))

-- Check that model_node depends on data_node
assert(expect_dependency(p, "data_node",
 ↪  "model_node"))

-- Check runtimes and serializers are correctly set
assert(expect_runtime(p, "model_node", "R"))
assert(expect_serializer(p, "model_node", ^pmml))
assert(expect_deserializer(p, "model_node", ^ipc))

-- Check that model_node was actually built
 ↪  successfully
assert(expect_computed(p.model_node))
```

These structural tests are most valuable as a guard against refactoring accidents. If you reorganise the pipeline and accidentally break a dependency, `expect_dependency` will catch it immediately, before any downstream output has a chance to silently produce wrong results.

4.4.6 The Fixture-Chain Pattern

For more complex test suites, T supports composing pipelines with `chain()` and `parallel()`. The pattern is to separate data-producing nodes from assertion nodes, and compose them into a combined pipeline:

```
-- A fixture pipeline produces the base data
fixture = pipeline {
  data = node(
    command = read_csv("tests/data/sample.csv"),
    serializer = ^csv
  )
}

-- A test pipeline consumes and transforms it, then
 ↪  checks the result
test_filter = pipeline {
  filtered = node(
    command = data |> filter($score >= 88),
    serializer = ^csv,
    deserializer = ^csv
  )
  -- A dedicated "check" node whose output is a Dict
 ↪  of assertion results
  check_filter = node(
```

```
    command = [
      nrow_check:    assert(expect_nrow(filtered,
 ↪  2)),
      colnames_check:
 ↪  assert(expect_colnames(filtered, ["name",
 ↪  "score", "grade"]))
    ],
    serializer = ^json,
    deserializer = ^csv
  )
}

-- chain() wires fixture.data into test_filter;
-- parallel() allows multiple independent test
 ↪  pipelines to run from the same fixture
combined = chain(fixture, parallel(test_filter))
build_pipeline(combined)
```

The `check_filter` node serializes its result as JSON: a named Dict where each key is the name of an assertion and each value is `true` on success or an `Error` on failure. This makes test results inspectable and diffable, not just pass/fail.

4.4.7 Summary of Testing Levels

In a well-tested T project, you will have tests at three levels:

Level	Where	Tool	What it catches
Function	`tests/test_that.t`, or `tests/test_*.py`	`tlt test` / `pytest`	Bugs in the R/Python helper functions used inside nodes

Level	Where	Tool	What it catches
Node output	`tests/test-*.t`	`t test` + `expect_*`	Wrong data shapes, unexpected NAs, bad column names after a build
Pipeline structure	`tests/test-*.t`	`t test` + `expect_pipeline`	Broken DAG structure, wrong runtimes/serializers, dependency mismatches

These three layers complement each other. Function tests are fast and cheap to write. Node output tests catch integration bugs that only appear when R and Python code runs inside a Nix sandbox. Pipeline structure tests catch architectural regressions when the pipeline definition is refactored.

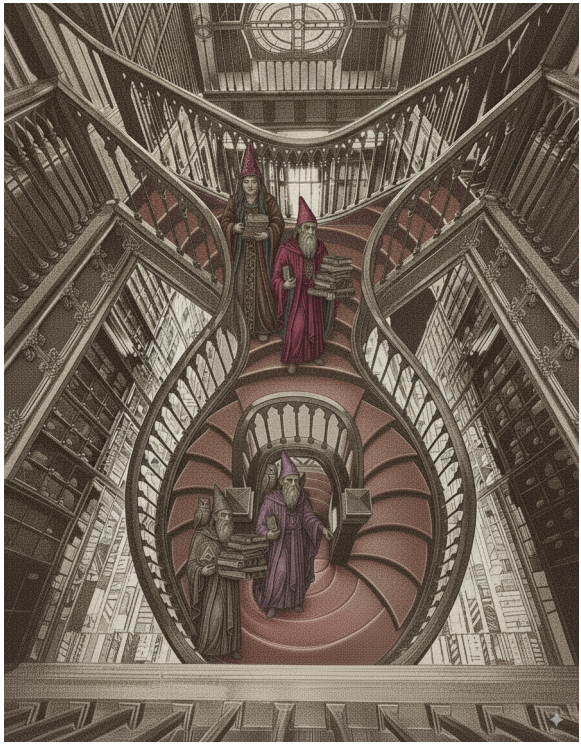
4.5 Summary

In this chapter, we covered the safety protocols of professional data science with T:

1. **Unit tests** (R's `{testthat}`, Python's `pytest`) prove that the helper functions living inside your nodes work correctly in isolation.
2. **Git workflows** (Trunk-Based Development, rebasing, interactive staging) keep collaboration clean and protect you from AI-generated regressions.
3. **Testcraft** (`t test`, `expect_*`) tests T pipelines at two levels: node outputs (does the data look right after a build?) and pipeline structure (does the DAG have the right shape, runtimes, and serializers?).

With these testing layers, Git hygiene, and test oracles in place, we now have the foundation to build an **AI Harness**. In the next chapter, we will see how to combine these verification tools with `AGENTS.md` and code-inspection literacy to safely delegate coding tasks to autonomous AI agents.

5 The AI Harness: AGENTS.md, Code Inspection, and Verification



5.1 Learning Outcomes

By the end of this chapter, you will:

- Understand the fundamental shift from **code author** to **code reviewer and harness architect** in AI-assisted data science.
- Master the **six core programming language traps** that Large Language Models (LLMs) consistently introduce in Python and R, and learn how to identify them by reading code.
- Understand what an `AGENTS.md` file is, why it is essential, and how it translates human code-review criteria into machine-enforceable negative constraints.
- Structure a production-grade `AGENTS.md` file covering environment contracts, language guardrails, data boundaries, and verification protocols.
- Combine `AGENTS.md`, Nix, and `t test --json` into a closed, autonomous self-healing agent loop anchored by strict test oracles.

5.2 Introduction: The Inverted Pyramid of Programming

For decades, learning computer programming followed a predictable progression: you memorized syntax, studied standard library functions, learned algorithms and data structures, and spent hundreds of hours practicing the mechanics of typing code into an editor.

The advent of modern code-generation models has completely inverted this pyramid.

Today, generating fifty lines of syntactically valid R or Python code takes three seconds. An LLM can write boilerplate data cleaning scripts, draft visualization code, or outline complex pipelines almost instantaneously. Because the barrier to *producing* code has dropped to zero, typing speed and syntax memorization are no longer the primary bottlenecks of software development.

Instead, the bottleneck has shifted entirely to **inspection, verification, and harness engineering**.

```
TRADITIONAL WORKFLOW                    AI-ASSISTED
WORKFLOW

  Writing & Typing      (80%)
Specification &        (30%)
                                        Harness
Design

  Review & Debugging    (15%)           AI
Generation          ( 5%)

  Harness & Ops         ( 5%)           Inspection
&           (65%)
                            Verification
```

An LLM is a probabilistic token predictor. It does not understand domain logic, sample variance, memory allocators, or causal inference. It produces code that *looks* like the millions of scripts

in its training corpus. In data science, code that "looks plausible" is exceptionally dangerous: it will run without raising an error, create a chart, output a table, and silently compute the wrong numbers.

If you cannot read code critically, an AI assistant does not make you ten times faster—it makes you produce ten times more bugs per hour.

To harness AI safely and efficiently, you need two complementary skills:

1. **Inspection Literacy (The Human Reviewer):** Knowing the specific language semantic traps that models consistently fall into, so you can spot them immediately when inspecting a diff.
2. **Harness Engineering (The AI Architect):** Constructing the external environment (`AGENTS.md`, Nix, pure T nodes, and test oracles) that constrains the model and forces it to verify its own work.

5.3 Part 1: Spotting AI Mistakes: The Six Language Semantic Traps

When an LLM produces buggy analytical code, it rarely fails because it forgot how to write an `if` statement or an arithmetic expression. It fails because of subtle, implicit language mechanics.

Here are the six classic language traps that LLMs fall into across Python and R, why they happen, and how to spot them by reading code.

5.3.1 Trap 1: Reference Semantics and In-Place Mutation

One of the most frequent bugs in AI-generated Python code is unintended in-place mutation.

In Python, variables are names bound to objects in memory. When you pass a DataFrame, list, or dictionary into a function, Python passes the *object reference*. If the function modifies that object, the changes leak back into the calling environment.

5.3.1.1 What the LLM writes:

```
def remove_outliers(df, column, threshold=3.0):
    # The LLM attempts to clean the data
    z_scores = (df[column] - df[column].mean()) /
 ↪  df[column].std()
    # BUG: inplace=True modifies the caller's
     ↪  DataFrame in memory!
    df.drop(df[z_scores.abs() > threshold].index,
 ↪  inplace=True)
    return df
```

5.3.1.2 Why it fails:

If you pass your pipeline's primary dataset into `remove_outliers(raw_data, "price")`, `raw_data` is permanently mutated in the caller's scope. Any other node or downstream analysis expecting the un-dropped data will silently receive the filtered version. In a DAG pipeline or an interactive session, this destroys the mathematical property of **pure functions** and breaks caching.

A related Python bug that LLMs generate constantly is the **mutable default argument**:

```
# BUG: The list `columns_to_log` is created ONCE
 ↪  when the function is defined.
# Subsequent calls share the exact same list in
 ↪  memory!
def record_metrics(metric_name, value, history=[]):
    history.append((metric_name, value))
    return history
```

5.3.1.3 How to spot it:

- Look for `inplace=True` in pandas methods (`.drop()`, `.fillna()`, `.reset_index()`). In modern, reproducible pipelines, `inplace=True` should be considered an anti-pattern.
- Look for direct attribute mutation: `df["new_col"] = ...` on an input argument rather than `df.assign(new_col=...)` or creating an explicit copy (`df = df.copy()`).
- In function definitions, check default arguments: defaults must be immutable (`None`, integers, strings, tuples), never `[]` or `{}`.

> **R and Copy-on-Modify**
>
> Standard R data frames use **copy-on-modify** semantics, which protects you from this specific bug in base R and `{tidyverse}`. However, if an LLM uses R environments, `data.table` (which uses `:=` for in-place modification), or `R6` reference classes, the exact same in-place mutation trap applies.

5.3.2 Trap 2: The "Notebook Hangover": Global Scope Capture

LLMs are trained on hundreds of thousands of Jupyter notebooks and exploratory scripts. In a notebook, developers frequently define variables in global scope (`df = pd.read_csv(...)`) and then write small helper functions that casually refer to those global names instead of passing them as arguments.

5.3.2.1 What the LLM writes:

```
raw_sales = pd.read_csv("sales.csv")

def calculate_regional_share(region_name):
    # BUG: `raw_sales` is NOT passed into the
     ↪ function!
    # It captures `raw_sales` from the global
     ↪ namespace.
```

```
    regional_total = raw_sales[raw_sales["region"]
↪   == region_name]["amount"].sum()
    grand_total = raw_sales["amount"].sum()
    return regional_total / grand_total
```

5.3.2.2 Why it fails:

This function works perfectly when you run it interactively in the notebook where `raw_sales` happens to exist. But the moment you move this function into a modular package (`src/utils.py`), a T pipeline node (`pyn()`), or an isolated unit test, it crashes with `NameError: name 'raw_sales' is not defined`. Even worse: if an outer variable with that name *does* exist in another script, the function runs, but reads the wrong data!

A subtle variant of this trap is **closure late-binding**:

```
# The LLM tries to generate a list of multiplier
↪  functions
multipliers = [lambda x: x * i for i in range(4)]

# You expect multipliers[2](10) to be 20.
# BUG: In Python, closures bind the variable name
↪  `i`, not its value at creation.
# All functions evaluate `i` when called (when i is
↪  3)!
print([m(10) for m in multipliers])
# Output: [30, 30, 30, 30] (NOT [0, 10, 20, 30]!)
```

5.3.2.3 How to spot it:

- Scrutinize the function body: does every variable name originate from an argument, an imported module, or a local assignment inside the function?
- If you see a variable that is neither a parameter nor locally defined, the LLM has captured outer scope.

5.3.3 Trap 3: Silent Type Coercion, Truthiness, and Falsy Edge Cases

Python and R handle type coercion and boolean evaluation ("truthiness") very differently, and LLMs constantly blend the rules of both.

5.3.3.1 What the LLM writes in Python:

```
def filter_active_customers(df):
    # BUG: In pandas, evaluating a Series or
     ↪  DataFrame as a boolean raises:
    # "ValueError: The truth value of a Series is
     ↪  ambiguous."
    if df["status"]:
        return df[df["status"] == "active"]
    return df
```

Or when checking for empty collections:

```
def process_weights(weights):
    # The LLM intends to check if weights is None or
     ↪  empty.
    # BUG: If weights is a numpy array of numbers,
     ↪  `if not weights:` raises
    # "ValueError: The truth value of an array with
     ↪  more than one element is ambiguous."
    # If weights is a float 0.0, `not weights`
     ↪  evaluates to True!
    if not weights:
        weights = default_weights()
```

5.3.3.2 What the LLM writes in R:

```
# Vector recycling trap
calculate_adjusted_scores <- function(scores,
 ↪  adjustments) {
  # If length(scores) is 10 and length(adjustments)
   ↪  is 3,
  # R silently recycles `adjustments` over `scores`!
  scores + adjustments
}
```

In R, if `adjustments` is shorter than `scores`, R repeats elements of `adjustments` to match the length of `scores`. Unless the lengths are not multiples of each other, this happens with zero warnings. The model assumes vectorized broadcasting behaves like NumPy, but R's recycling rule silently computes nonsensical calculations.

Another classic R trap is confusing `NULL`, `NA`, and empty vectors:

```
x <- NULL
length(x) == 0    # TRUE
is.na(x)          # logical(0) -> will cause `if
 ↪  (is.na(x))` to crash with "argument is of length
 ↪  zero"!
```

5.3.3.3 How to spot it:

- In Python: Look for `if df:` or `if series:`. Insist on explicit checks: `if df.empty:` or `if len(df) == 0:`. Check `if x is None:` instead of `if not x:`.
- In R: Check vector arithmetic between two vectors of uncertain lengths. Insist on `stopifnot(length(scores) == length(adjustments))` or explicit column operations inside `{dplyr}` / `colcraft`.

5.3.4 Trap 4: Floating-Point Comparisons & Numerical Tolerance

Computers represent real numbers using binary floating-point representation (IEEE 754). Many decimal fractions (like 0.1 or 0.2) cannot be represented with finite binary precision, leading to minute rounding discrepancies.

5.3.4.1 What the LLM writes:

```
def test_budget_allocation():
    allocations =
 ↪  calculate_allocations(budget=1000.0)
    # BUG: Floating point arithmetic: 0.1 + 0.2 =
     ↪  0.30000000000000004
    # This assertion will fail intermittently
     ↪  depending on CPU architecture!
    assert allocations["marketing"] +
     ↪  allocations["rd"] + allocations["sales"] ==
     ↪  1000.0
```

```
# R equivalent
if (sum(proportions) == 1.0) {
  # BUG: Might evaluate to FALSE even when the math
   ↪  is correct!
  proceed_with_analysis()
}
```

5.3.4.2 Why it fails:

Direct equality comparisons (==) on floating-point results are brittle. A calculation that succeeds on an Apple Silicon laptop may fail on an x86 Linux GitHub Actions runner by 10^{-16}.

5.3.4.3 How to spot it:

- Search for == or != applied to continuous numerical columns or float calculations.

- Replace them with tolerance-based comparisons:
  - In Python: `math.isclose(a, b, rel_tol=1e-9)` or `np.isclose(a, b)`.
  - In R: `isTRUE(all.equal(a, b))` or `expect_equal(a, b)`.
  - In T testcraft: `expect_equal(a, b)`.

5.3.5 Trap 5: The "Helpful" LLM Syndrome: Silent Error Swallowing

LLMs are tuned to be helpful and avoid crashes. When an LLM encounters potential failure points (like missing files, invalid JSON, or parse errors), its instinct is often to catch the exception and return a default value or `None`.

5.3.5.1 What the LLM writes:

```
def extract_transaction_amount(record):
    try:
        amount =
 ↪  float(record["transaction"]["amount"])
        currency = record["transaction"]["currency"]
        return convert_to_eur(amount, currency)
    except Exception:
        # BUG: Swallows KeyErrors, TypeErrors,
         ↪  ZeroDivisionError,
        # and network failures, returning None!
        return None
```

5.3.5.2 Why it fails:

In a reproducible analytical pipeline, **silent failures are fatal**. If the data provider changes the key name from `"amount"` to `"total_amount"`, this function will not throw an error. Instead, it will silently return `None` for every single row.

Downstream nodes will proceed to calculate summary statistics, compute averages over non-null values, and produce an empty or distorted report. The pipeline finishes with exit code 0, reporting a clean build when the entire analysis was actually wiped out.

5.3.5.3 How to spot it:

- Look for `except Exception:` or bare `except:`.
- Look for `tryCatch(..., error = function(e) NULL)` in R.
- Defensive rule: Exceptions should only catch specific, anticipated errors (like `FileNotFoundError`), and unexpected errors must **fail loudly and immediately**.

5.3.6 Trap 6: API Hallucination and Deprecation Drift

Because LLMs are trained on historical internet data, their training corpora include documentation and tutorials spanning decades. Consequently, models frequently mix old, deprecated APIs with modern code, or synthesize methods that "feel" like they should exist.

5.3.6.1 What the LLM writes:

```
# In modern pandas (2.0+), .append() was removed!
# The LLM still writes it because it saw it
 ↪  thousands of times in 2018 tutorials.
summary_df = summary_df.append(new_row,
 ↪  ignore_index=True)
```

```
# API hallucination: invent plausible-sounding
 ↪  parameters
df = pd.read_csv("large_data.csv",
 ↪  optimize_memory=True, fast_mode=True)
```

5.3.6.2 How to spot it:

- Check parameters against official documentation if a method signature looks unfamiliar.
- Rely on your Nix environment: if an LLM hallucinated an old pandas 1.x method, running `t test` or `nix develop` will immediately expose the `AttributeError`.

5.4 Part 2: Building the Harness: The Role and Structure of `AGENTS.md`

Now that you know the six traps to look for, consider the human cost of catching them.

If every time an AI generates a node or test you have to manually remind it not to use `inplace=True`, not to swallow exceptions, and to use tolerance for float checks, you are wasting cognitive energy on repetitive corrections.

This is where **`AGENTS.md`** enters.

5.4.1 What is `AGENTS.md`?

`AGENTS.md` is an open standard and convention for repository-level agent guidance. Modern AI developer tools—including Cursor, Claude Code, GitHub Copilot, and T's built-in agents—automatically search for and ingest `AGENTS.md` in the project root before formulating answers or generating code.

It is **system prompt engineering persisted as code**. Instead of giving instructions in an ephemeral chat window that vanishes when you close your browser, `AGENTS.md` lives in your Git repository, version-controlled alongside your code.

```
THE AI HARNESS


1. CONTEXT: AGENTS.md & T-LANGUAGE-REFERENCE.md

   (Domain rules, schemas, negative constraints)



2. CONSTRAINTS: Nix & Pure T DAG Nodes
```

```
   (Isolated sandboxes, immutable Arrow IPC)


3. ORACLES: Golden Datasets & Testcraft

   (Objective mathematical ground truth)
```

5.4.2 The Strategy: Translating Inspection Traps into Negative Constraints

The secret to writing an effective `AGENTS.md` is to avoid vague advice like *"Write clean, readable code"*. Models already know how to write clean-looking code; what they need are **explicit negative constraints** that prohibit their habitual failure modes.

Every language trap we covered in Part 1 can be translated into an unambiguous rule:

Language Trap	Instruction in `AGENTS.md`
In-place mutation	*"Never use `inplace=True`. Always return a transformed copy of data."*
Global scope capture	*"Functions must be pure: all data must enter via parameters; no global variable reads."*
Float equality	*"Never compare floats with `==`. Always use `isclose` or `expect_equal` with tolerance."*

Language Trap	Instruction in `AGENTS.md`
Error swallowing	*"Never catch generic `Exception` to return `None`. Let errors raise or fail loudly."*
Ad-hoc installs	*"Never invoke `pip install` or `install.packages()`. All packages are managed via Nix."*

5.5 An Annotated, Complete `AGENTS.md` Template

Below is a production-ready `AGENTS.md` tailored for a reproducible polyglot project using T, Nix, Python, and R. You can adapt this template directly for your course project.

```
# Agent Guide: Polyglot Data Pipeline Project

This document provides mandatory operational
 ↪  context, constraints, and coding standards
for AI agents and human contributors working in this
 ↪  repository.

---

## 1. Environment & Execution Contract (Nix-First)

- **Hermetic Sandbox**: All execution must occur
 ↪  inside the Nix development shell (`nix
 ↪  develop`).
```

```
- **No Ad-Hoc Package Managers**: NEVER invoke `pip
 ↪  install`, `install.packages()`, `conda`,
  or `apt-get`. All dependencies are declared in
 ↪  `flake.nix` and `tproject.toml`.
- **Reproducibility Guarantee**: If a required
 ↪  package is missing, instruct the user to add it
  to `flake.nix` rather than installing it
 ↪  imperatively.

---

## 2. Architecture: Pure Functional DAG Pipelines

- **Orchestration**: All data transformations must
 ↪  be expressed as nodes in `src/pipeline.t`.
- **Pure Nodes**: Every node must take explicit
 ↪  inputs and produce exactly one serialized
 ↪  output.
- **Data Interchange**: Pass tabular data between R,
 ↪  Python, and Julia using Apache Arrow IPC
 ↪  (``^ipc`).
  Do NOT export ad-hoc CSV files or custom temporary
 ↪  files.
- **Read-Only Data**: The `data/` directory is
 ↪  strictly immutable. Never attempt to write,
  delete, or overwrite files in `data/`. All
 ↪  generated assets belong in pipeline node
 ↪  storage.

---

## 3. Language Semantic Guardrails (Mandatory)
```

```
### Python Standards
- **Immutability**: Do NOT use `inplace=True` on
 ↪  pandas objects. Return new DataFrames via
  `.assign()`, `.copy()`, or functional
   ↪  transformations.
- **No Mutable Defaults**: Never define functions
 ↪  with mutable default arguments (e.g., `def
 ↪  f(x=[])`).
- **Strict Scoping**: Helper functions must be pure.
 ↪  Do not reference variables from outer/global
  scope; pass all inputs explicitly as arguments.
- **Fail Loudly**: Never use broad `try: ... except
 ↪  Exception:` blocks that return `None`. If an
  unexpected error occurs, allow it to raise.
- **Numerical Tolerance**: Never assert direct float
 ↪  equality (`==`). Use `pytest.approx()`
  or `np.isclose()`.

### R Standards
- **Pure Functions**: Write modular functions inside
 ↪  `R/`. Avoid modifying global options
 ↪  (`options()`)
  or working directories (`setwd()`).
- **Explicit Length Checks**: Guard against vector
 ↪  recycling bugs by verifying vector lengths
 ↪  before
  element-wise operations (`stopifnot(length(a) ==
 ↪  length(b))`).
- **Defensive Missingness**: Explicitly account for
 ↪  `NA` and `NULL` values. Never assume a vector
  is non-empty without checking.

---
```

```
## 4. Verification Protocol & Oracles

- **Test Suite**: Before declaring any task
 ↪  complete, you MUST execute the project test
 ↪  suite:
  ```bash
  t test --json
```

- **Exit Criteria**: A task is only complete when `status` is `"passed"` with 0 failures.
- **Integrity of Oracles**: You are STRICTLY FORBIDDEN from modifying test assertions or golden datasets in `tests/` to make failing code pass. If a test fails against an oracle, the bug is in your implementation.
- **Golden Snapshots**: When refactoring data wrangling nodes, verify output equivalence against the verified reference datasets in `tests/fixtures/`.

```
---

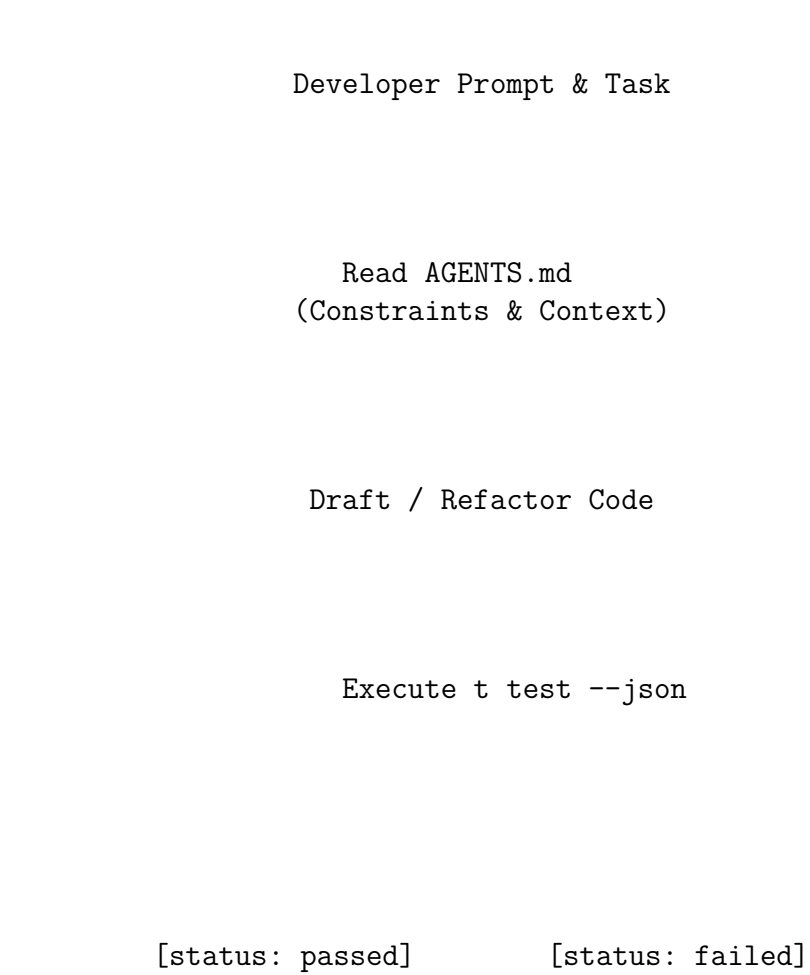
## Part 3: The Closed-Loop Autonomous Agent

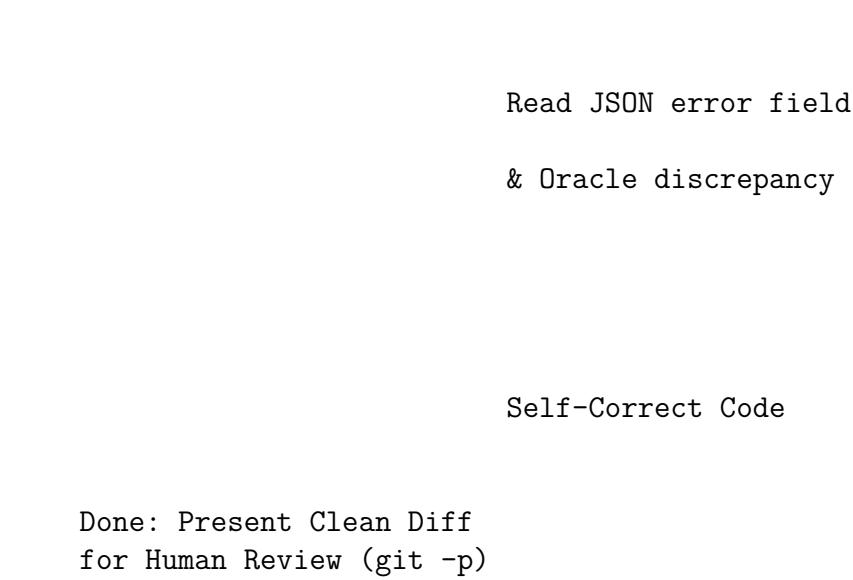
When you provide both an `AGENTS.md` and an
automated testing oracle, you unlock the ability to
run **closed-loop autonomous agent cycles**.

In a standard chat workflow, you are trapped in a
tedious human-in-the-loop ping-pong:
1. You prompt the model.
2. The model outputs code.
3. You copy it into your editor.
```
```

4. You run it and get an error.
5. You paste the error back to the model.
6. The model apologizes and tries again.

In a closed-loop harness, the AI agent interacts directly with your project's CLI:





Notice why **`t test --json`** and **test oracles** are critical to this loop:

1. **Machine-Readable Diagnostics:** The agent does not need to parse noisy terminal ANSI colors. The structured JSON gives it the exact test file, line number, and expectation failure.
2. **Objective Ground Truth:** Because the rule in `AGENTS.md` prohibits modifying the oracle in `tests/`, the agent cannot cheat by watering down the test. It must fix its analytical logic until the math agrees with the golden benchmark.

```
3. **Effortless Human Oversight:** You only inspect
the work once the agent has proven that all oracles
pass. You then run `git add -p` to review the final
diff for stylistic elegance and architectural
cleanliness.

### Model Tiering: Frontier Reasoners for
Architecture, Flash Models for Implementation

A practical trap when building agentic harnesses is
treating all language models identically. Using a
heavy, slow reasoning model to patch a one-line
syntax error is inefficient and burns through API
quotas rapidly. Conversely, relying on a lightweight
model to draft complex DAG topology or derive
mathematical invariants often results in subtle
structural flaws.

In production engineering, adopting a **two-tier
model strategy** dramatically improves both velocity
and output quality:

1. **Frontier & Reasoning Models (The Architect &
Auditor):**
   - **Role:** Planning, DAG architecture, authoring
   `AGENTS.md` constraints, and deep code review.
   - **When to use:** Use models known for high
   reasoning capability (such as Claude Sonnet,
   OpenAI o-series/GPT-4o, DeepSeek Reasoner, or
   Gemini Pro) when framing the problem,
   establishing data contracts between pipeline
   nodes, diagnosing stubborn test failures, and
   conducting the final diff inspection.
2. **Fast & "Flash-Like" Models (The Builder):**
```

```
   - **Role:** Code synthesis, fast test iteration
   loops, and local refactoring.
   - **When to use:** Once the interfaces and test
   oracles are firmly defined in your repository,
   switch to a nimbler, low-latency model (such as
   Gemini Flash, Claude Haiku, or DeepSeek Flash).
   Because the search space is constrained by your
   tests and `AGENTS.md`, a fast model can iterate
   through the edit-test cycle in seconds without
   losing coherence.

By decoupling **architectural reasoning** from
**iterative code generation**, you eliminate latency
bottlenecks while preserving rigorous software
standards.

---

## Summary

The rise of AI code generation does not diminish the
need for foundational computer science knowledge; it
sharpens it.

1. **Code Review is the New Core Skill:** Because
drafting code is cheap, your primary value as a data
scientist is the ability to inspect, verify, and
validate AI output.
2. **Watch for the Six Traps:** In-place mutation,
global scope leakage, truthiness confusion, float
equality, silent error swallowing, and API
hallucinations are the classic signatures of AI
code.
```

```
3. **`AGENTS.md` is System Engineering:** Encode
your review standards as persistent,
version-controlled constraints so the model avoids
these traps automatically.
4. **Harness + Oracles = Autonomy:** When you
surround an AI model with Nix (environment), T (pure
DAG execution), `AGENTS.md` (constraints), and
testcraft (oracles), you transform an erratic
chatbot into a reliable, autonomous engineering
partner.
5. **Tier Your Models:** Use top-tier reasoning
models for planning architecture and auditing code;
delegate fast inner loops and boilerplate generation
to quick flash models.

---

## Exercises

### Exercise 1: Spotting Semantic Traps

Inspect the following AI-generated Python function
intended to compute normalized min-max scaling for
feature columns in a dataset:

```python
import pandas as pd

def min_max_scale_data(df, columns,
feature_range=(0, 1)):
    low, high = feature_range
    for col in columns:
        try:
            col_min = df[col].min()
```
```

```
            col_max = df[col].max()
            if col_min == col_max:
                df[col] = low
            else:
                df[col] = low + ((df[col] - col_min)
                / (col_max - col_min)) * (high -
                low)
        except Exception:
            df[col] = None
    return df
```

Questions:

1. Identify at least **three** of the six language semantic traps discussed in Part 1 present in this function.
2. Explain specifically how the `try...except` block could mask a catastrophic pipeline failure.
3. Rewrite the function so that it is pure, preserves immutability, fails loudly on invalid inputs, and does not mutate the caller's DataFrame.

5.5.1 Exercise 2: Authoring an `AGENTS.md`

In your course repository, create an `AGENTS.md` file in the root directory.

Include: 1. Clear instructions designating `data/` as read-only. 2. Negative constraints forbidding `inplace=True` and unhandled exceptions. 3. The exact CLI command the agent must run to verify changes (`t test --json`). 4. An instruction forbidding the agent from altering files in `tests/`.

Test your setup by prompting an AI assistant to write a new pipeline node in `src/` and instructing it to obey `AGENTS.md`. Verify whether it respects the constraints!

6 A Short Intro to Packaging Your Code in R and Python



What you'll learn by the end of this chapter:

- Why organizing your code into a formal package is the ultimate form of reproducibility and reusability.

- How to create, document, and test a basic R package using `{devtools}` and `{usethis}`.
- How to create, document, and test a modern Python package using `uv` and `pytest`.
- How to install your own packages directly from GitHub, enabling you to share your tools with colleagues and your future self.

6.1 Introduction: Why Bother Packaging?

So far, we have built a robust workflow based on three pillars: reproducible environments with Nix, reproducible history with Git, and reproducible logic with functional programming. We've organized our code into functions, which are a massive improvement over messy scripts.

The final, logical step in this journey is to treat our collection of functions not just as a set of helper scripts, but as a formal **package**. A package is more than just a folder of code; it's a self-contained, distributable, and installable unit of software that bundles together code, data, documentation, and tests.

You might think, "I'm a data scientist, not a software engineer. Isn't this overkill?" The answer is a definitive **no**. Packaging your code, even for an internal analysis project, provides enormous benefits:

1. **Reusability:** Instead of copying and pasting your `clean_data()` function from project to project, you can simply `import mypackage` or `library(mypackage)` and use a single, trusted version.

2. **Distribution & Collaboration:** How do you share your work with a colleague? Emailing a zip file of scripts is a recipe for disaster. Sending them a single command—`devtools::install_github("my_repo")`—is robust and professional.

3. **Documentation:** Packaging forces you into a standardized way of documenting your functions. This makes your code understandable to others and, more importantly, to yourself six months from now.

4. **Testing:** A package provides a formal framework for running unit tests, ensuring that your functions work as expected and giving you the confidence to make changes without breaking things.

5. **Dependency Management:** A package explicitly declares all of its dependencies (e.g., "this package needs `dplyr` version 1.1.0 or newer"). This solves a huge source of reproducibility errors.

In this chapter, we will walk through the process of creating a simple package in both R and Python. The goal is not to become an expert package developer, but to understand the structure and benefits so you can apply this powerful "packaging mindset" to all your future projects.

6.2 Part 1: Creating an R Package with {usethis} and {devtools}

The R community has developed an outstanding set of tools that make package development incredibly streamlined. The two essential packages are:

- **{devtools}:** Provides core development tools like `install()`, `test()`, and `check()`.
- **{usethis}:** A workflow package that automates all the boilerplate. It creates files, sets up infrastructure, and guides you through the process.

Let's build a package called `cleanR`, which will contain a function to standardize column names.

6.2.1 Step 1: Project Setup

First, make sure you have the necessary tools installed: `install.packages(c("devtools", "usethis", "roxygen2"))`.

Now, let `{usethis}` create the package structure for you. From your R console, run:

```
usethis::create_package("~/Documents/projects/cleanR")
```

This will create a new `cleanR` directory with all the necessary files and subdirectories. It will also open a new RStudio session for that project. The key components are:

- `R/`: This is where your R source code files will live.
- `DESCRIPTION`: A metadata file describing your package, its author, license, and dependencies.
- `NAMESPACE`: A file that declares which functions your package exports for users and which functions it imports from other packages. **You should never edit this file by hand.** `{roxygen2}` will manage it for you.

6.2.2 Step 2: Write and Document a Function

Let's create our function. {usethis} helps with this too:

```
usethis::use_r("clean_names")
```

This creates a new file R/clean_names.R and opens it for editing. Let's add our function, including special comments for documentation. These #' comments are used by the {roxygen2} package to automatically generate the official documentation.

```
# In R/clean_names.R

#' Clean and Standardize Column Names
#'
#' This function takes a data frame and returns a
 ↪  new data frame with
#' cleaned-up column names (lowercase, with
 ↪  underscores instead of spaces
#' or periods).
#'
#' @param df A data frame.
#' @return A data frame with standardized column
 ↪  names.
#' @export
#' @examples
#' messy_df <- data.frame("First Name" = c("Ada",
 ↪  "Bob"), "Last.Name" = c("Lovelace", "Ross"))
#' clean_names(messy_df)
clean_names <- function(df) {
  old_names <- names(df)
  new_names <- tolower(old_names)
  new_names <- gsub("[ .]", "_", new_names)
```

```
  names(df) <- new_names
  return(df)
}
```

The key tags here are:

- `@param`: Describes a function argument.
- `@return`: Describes what the function returns.
- `@export`: This is crucial. It tells R that you want this function to be available to users when they load your package with `library(cleanR)`.
- `@examples`: Provides runnable examples that will appear in the help file.

Now, run the magic command to process these comments:

```
devtools::document()
```

This updates the `NAMESPACE` file and creates the help file (`man/clean_names.Rd`). You can now see your function's help page with `?clean_names`.

6.2.3 Step 3: Add Unit Tests

A package without tests is a package waiting to break. `{usethis}` makes setting up tests trivial.

```
usethis::use_testthat() # Sets up the
 ↪  tests/testthat/ directory
usethis::use_test("clean_names") # Creates
 ↪  tests/testthat/test-clean_names.R
```

Now, edit the test file to add your expectations.

```
# In tests/testthat/test-clean_names.R
test_that("clean_names works with spaces and
 ↪  periods", {
  messy_df <- data.frame("First Name" = c("A"),
 ↪  "Last.Name" = c("B"))
  cleaned_df <- clean_names(messy_df)

  expected_names <- c("first_name", "last_name")

  expect_equal(names(cleaned_df), expected_names)
})

test_that("clean_names handles already clean names",
 ↪  {
  clean_df <- data.frame(a = 1, b = 2)
  # The function should not change anything
  expect_equal(names(clean_names(clean_df)), c("a",
   ↪  "b"))
})
```

To run all the tests for your package, use:

```
devtools::test()
```

6.2.4 Step 4: Check and Install

The final step before sharing is to run the official R CMD check, the gold standard for package quality. This command runs all tests, checks documentation, and looks for common problems.

```
devtools::check()
```

If your package passes with 0 errors, 0 warnings, and 0 notes, you are in great shape.

Now, let's install it locally.

```
devtools::install()
```

You can now use your package in any R session with `library(cleanR)`.

6.2.5 Step 5: Install from GitHub

To share your package, the easiest way is with GitHub.

1. Create a new, empty repository on GitHub (e.g., `cleanR`).
2. In your local project, follow the instructions GitHub provides to link your local repository and push your code. This usually involves commands like:

   ```
   git remote add origin
    ↪  git@github.com:yourusername/cleanR.git
   git branch -M main
   git push -u origin main
   ```

3. Now, anyone (including you on a different machine) can install your package with a single command:

   ```
   # You might need to install {remotes} first
   # install.packages("remotes")
   remotes::install_github("yourusername/cleanR")
   ```

Congratulations, you have created and shared a fully functional R package!

6.3 Part 2: Creating a Minimal Python Package with uv

The Python packaging ecosystem is rapidly modernizing. While we use Nix to manage our overall environment, we still need to define the metadata and structure for our Python package. We will use **uv**, an extremely fast and modern tool, for one specific purpose: initializing our project's configuration file. We will **not** use `uv` to manage a virtual environment, as Nix already handles that for us.

Let's build a Python package called `pyclean`, the equivalent of our R package.

6.3.1 Step 1: Project Setup with uv

First, ensure `uv` is installed in your Nix environment. Then, create a directory for your new package and initialize it:

```
mkdir pyclean
cd pyclean
uv init --bare
```

The `--bare` flag is perfect for our Nix workflow. It creates only the essential `pyproject.toml` file without creating a virtual environment or extra directories. This leaves us with a clean slate.

Now, we must create the source and test directories manually. We'll use the standard `src` layout:

```
mkdir -p src/pyclean
mkdir tests
touch src/pyclean/__init__.py
```

Your project structure should now look like this (check it using the `tree` command):

```
pyclean/
   pyproject.toml
   src/
      pyclean/
          __init__.py
   tests/
```

6.3.2 Step 2: Write a Function and Declare Dependencies

Let's create our `clean_names` function inside a new file, `src/pyclean/formatters.py`.

```
# In src/pyclean/formatters.py
import pandas as pd

def clean_names(df: pd.DataFrame) -> pd.DataFrame:
    """Clean and standardize column names of a
     ↪ DataFrame.

    Args:
        df: The input pandas DataFrame.
```

```
    Returns:
        A pandas DataFrame with standardized column
↪   names.
    """
    new_df = df.copy()
    new_cols = {col: col.lower().replace(" ",
↪   "_").replace(".", "_") for col in
↪   new_df.columns}
    new_df = new_df.rename(columns=new_cols)
    return new_df
```

To make this function easily importable, we expose it in `src/pyclean/__init__.py`:

```
# In src/pyclean/__init__.py
from .formatters import clean_names

__version__ = "0.1.0"
```

Next, we must declare our dependencies by manually editing `pyproject.toml`. We need `pandas` for our function and `pytest` for our tests.

```
# In pyproject.toml
[project]
name = "pyclean"
version = "0.1.0"
description = "A simple package to clean data."
dependencies = [
    "pandas>=2.0.0",
]
```

```
[project.optional-dependencies]
test = [
    "pytest",
]

[tool.pytest.ini_options]
pythonpath = [
  "src"
]
```

The `pythonpath = ["src"]` line is the key. Without it, you'd first need to install your `pyclean` library in editable mode using pip before running the tests. By adding this block, simply running `pytest` from the command line will work.

6.3.3 Step 3: Add Unit Tests

Create a new test file, `tests/test_formatters.py`, and add your tests.

```
# In tests/test_formatters.py
import pandas as pd
from pyclean import clean_names

def test_clean_names_happy_path():
    messy_df = pd.DataFrame({"First Name": ["Ada"],
 ↪  "Last.Name": ["Lovelace"]})
    cleaned_df = clean_names(messy_df)
    expected_cols = ["first_name", "last_name"]
    assert list(cleaned_df.columns) == expected_cols
```

```
def test_clean_names_is_idempotent():
    clean_df = pd.DataFrame({"first_name": ["a"],
 ↪  "last_name": ["b"]})
    still_clean_df = clean_names(clean_df)
    assert list(still_clean_df.columns) ==
     ↪  list(clean_df.columns)
```

Since your Nix environment provides all the tools, you can run tests directly from your terminal:

```
pytest
```

6.3.4 Step 4: Build and Install

To package your code, you need a build tool. It turns out that `uv` bundles a build tool with it, so we only need to call `uv build`:

```
# In your terminal, from the root of the 'pyclean'
 ↪  project
uv build
```

This creates a `dist/` directory containing a source distribution (`.tar.gz`) and a compiled wheel (`.whl`). The wheel is the modern standard for distribution.

Outside of a Nix shell, to use your package during development, you can install it in "editable" mode. This creates a link to your source code, so any changes you make are immediately reflected without needing to reinstall.

```
# Install the package and its test dependencies
pip install -e .[test]
```

But we are working from a Nix shell. Instead, we will simply edit our `default.nix` to update the `PYTHONPATH` environment variable, so our package can easily be found. If you look at the `default.nix` file of the course you've been using, you'll see the following at the bottom:

```
shellHook = ''
  export PYTHONPATH=$PWD/pyclean/src:$PYTHONPATH
'';
```

(you may need to adapt the path depending on where you're developing the package). With this, dropping into the Nix shell, starting the Python interpreter and then typing `import pyclean` will work without any issues.

6.3.5 Step 5: Install from GitHub

Sharing via GitHub is the most common way to distribute packages that aren't on the official Python Package Index (PyPI):

1. Create a new, empty repository on GitHub.
2. Push your local project to the remote repository.
3. Now, anyone can install your package directly from GitHub using `pip`, which is smart enough to find and process your `pyproject.toml` file: `bash pip install git+https://github.com/yourusername/pyclean.git`

For Nix environments, add this to your `default.nix`:

```
pyclean = pkgs.python313Packages.buildPythonPackage
rec {
  pname = "pyclean";
  version = "0.1.0";
  src = pkgs.fetchgit {
    url = "https://github.com/b-rodrigues/pyclean";
    rev =
    "174d4d482d400536bb0d987a3e25ae80cd81ef3c";
    sha256 =
    "sha256-xTYydkuduPpZsCXE2fv5qZCnYYCRoNFpV7lQBM3LMSg=";
  };
  pyproject = true;
  propagatedBuildInputs = [
  pkgs.python313Packages.pandas
  pkgs.python313Packages.setuptools ];
  # Add more dependencies to propagatedBuildInputs
  as needed
};
```

You need to add the `rev`, which corresponds to the commit that want, and the `sha256`. To find the right `sha256`, start with an empty one (`sha256 = "";`) and try to build the package. The error message will give you the right `sha256`. Also not that this isn't the the most idiomatic way to build a Python package for Nix, but it's good enough for our purposes.

Finally, add `pyclean` to the `buildInputs` of the shell:

```
  buildInputs = [ rpkgs pyconf pyclean tex
  system_packages github_pkgs ];
```

This process is naturally more involved than simply calling `pip install`, but it has the advantage of being entirely reproducible.

6.4 Conclusion: The Packaging Mindset in the Age of AI

You have now successfully created, tested, documented, and shared a basic package in both R and Python. While there is much more to learn about advanced package development, you have already mastered the most important part: the **packaging mindset**.

From now on, when you start a new analysis project, think of it as a small, internal package.

- Put your reusable logic into functions.
- Place those functions in the `R/` or `mypackage/` source directory.
- Document them.
- Write a few simple tests to prove they work.
- Manage dependencies formally in `DESCRIPTION` or `pyproject.toml`.

Adopting this structure will make your work more robust, easier to share, and fundamentally more reproducible. It is the bridge between writing one-off scripts and building reliable, professional data science tools.

This packaging mindset becomes even more powerful when you introduce a modern collaborator: the LLM. The structured, component-based nature of a package is the perfect way to interact with AI assistants.

A package provides a clear contract and a well-defined structure that LLMs thrive on. Instead of a vague prompt like, "Refactor my messy analysis script," you can now make precise, targeted requests:

- "Here is my function `clean_names`. Please write three `pytest` unit tests for it, including one for the happy path, one for an empty DataFrame, and one for names that are already clean."
- "Generate the roxygen2 documentation skeleton for this R function, including `@param`, `@return`, and `@examples` tags."
- "I need a function in my `pyclean/utils.py` module that calculates the Z-score for a pandas Series. Please generate the function and its docstring."

This synergy is a two-way street. Not only does the structure help you write better prompts, but LLMs excel at generating the very boilerplate that makes packaging robust. Tedious tasks like writing standard documentation headers, creating skeleton unit test files, or even generating a first draft of a function based on a clear description become near-instantaneous.

This elevates your role from a writer of code to an **architect and a reviewer**. Your job is to design the components (the functions), prompt the LLM to generate the implementation, and then—most critically—use the testing framework you just built to rigorously verify that the AI-generated code is correct, efficient, and robust. You are the final authority, and the package structure gives you the tools to enforce quality control.

By combining the discipline of packaging with the power of LLMs, you lower the barrier to adopting best practices like comprehensive testing and documentation. This combination doesn't just make you faster; it makes you a more reliable and professional data scientist, capable of producing tools that are truly reproducible and built to last.

While a full guide to package development is beyond the scope of this course, it is the natural next step in your journey as a data

scientist who produces reliable tools. When you are ready to take that step, here are the definitive resources to guide you:

- **For R:** The "R Packages" (2e) book by Hadley Wickham and Jennifer Bryan is the essential, comprehensive guide. It covers everything from initial setup with `{usethis}` to testing, documentation, and submission to CRAN. **Read it online here.**
- **For Python:** The official **Python Packaging User Guide** is the place to start. For a more modern and streamlined approach that handles dependency management and publishing, many developers use tools like **Poetry** or **Hatch**.

Treating your data analysis project like a small, internal software package, complete with functions and tests, is a powerful mindset that will elevate the quality and reliability of your work.

6.5 Integrating Custom Packages into T Pipelines

Once your custom functions are packaged (either as an R package or a Python wheel installed into your Nix environment), using them in your pipeline is exceptionally clean. Instead of sourcing loose script files with `functions = ["src/utils.R"]`, your nodes can simply import your library:

```
p = pipeline {
  -- R node using packaged library
  cleaned_r = rn(
    command = <{
      library(cleanR)
```

```
      cleaned_r <- clean_names(raw_data)
    }>,
    deserializer = ^ipc,
    serializer = ^ipc
  )

  -- Python node using packaged library
  stats_py = pyn(
    command = <{
      from statstools.calculations import
↪   descriptive_stats
      stats_py =
↪   descriptive_stats(cleaned_r["price"].tolist())
    }>,
    deserializer = ^ipc,
    serializer = ^json
  )
}
```

This enforces the ultimate separation of concerns: - **Your packages**: Encapsulate domain logic, data cleaning, and statistical models—with their own unit tests and versioning. - **Nix**: Hermetically builds and provides your packages alongside language runtimes. - **T**: Declares the DAG, orchestrates execution, handles serialization, and caches intermediate artifacts.

6.6 Hands-on Exercises

6.6.1 Exercise 1: Build Your First Python Package with Nix

Goal: To build a complete, testable, and documented Python package and make it available in a reproducible Nix environment.

You will create a small Python package called `statstools`. This package will contain a single function that calculates descriptive statistics for a list of numbers.

Requirements:

1. **Project Structure:** Create the following directory structure for your package:

```
statstools/
   pyproject.toml
   default.nix
   src/
       statstools/
           __init__.py
           calculations.py
   tests/
        test_calculations.py
```

2. **Functionality:** In `src/statstools/calculations.py`, create a function `descriptive_stats(numbers: list) -> dict`. This function should take a list of numbers and return a dictionary containing the mean and standard deviation. Use the `numpy` library for the calculations.

3. **Documentation:** Write a clear docstring for your `descriptive_stats` function explaining what it does, its parameters (`Args`), and what it returns (`Returns`).

4. **Dependencies:** In your `pyproject.toml` file:

    - Define the project name as `statstools` and give it a version of `0.1.0`.
    - Add `numpy` as a runtime dependency.
    - Add `pytest` as an optional dependency for testing.
    - Add the `[tool.pytest.ini_options]` block to set the `pythonpath` correctly so `pytest` can find your `src` directory.

5. **Unit Tests:** In `tests/test_calculations.py`, write at least two tests for your function using `pytest`:

    - A "happy path" test with a simple list like `[1, 2, 3, 4, 5]`.
    - A test with negative numbers and floats.

6. **GitHub Repository:** Push your completed package to a new public repository on GitHub.

7. **Nix Expression:** This is the most critical part. Create a `default.nix` file in the root of your project. This file should:

    - Fetch your package's source code from your GitHub repository using `pkgs.fetchgit`. You will need to get the commit hash (`rev`) and the `sha256` hash (remember, you can get the correct `sha256` from the error message when you first try to build with an empty `sha256 = "";`).

- Define your package using `pkgs.python3Packages.buildPython`. Ensure you set `pyproject = true;` and list its dependencies (`numpy`, `pytest`) in `propagatedBuildInputs`.
- Create a reproducible shell using `pkgs.mkShell` that includes both your `statstools` package and the Python interpreter.
- **Hint:** Look closely at the `default.nix` file from the main course repository for a complete example of how a Python package is fetched from GitHub and built.

To verify your work: * Run `pytest` from within the Nix shell to ensure all your tests pass. * Start a Python interpreter inside the Nix shell and successfully run `from statstools import descriptive_stats`.

6.6.2 Exercise 2: Build and Package an R Tool with Nix

Goal: To apply R's best practices for package development (`usethis`, `devtools`, `roxygen2`) and integrate the final product into a reproducible Nix environment.

You will create a small R package called `datasummary`. This package will provide a function to quickly summarize a data frame.

Requirements:

1. **Project Setup:** Use `usethis::create_package("datasummary")` to generate the standard R package structure.
2. **Functionality:**
   - Use `usethis::use_r("summarize_df")` to create a new file for your function.

- The function, `summarize_df(df)`, should take a data frame as input.
- It should return a new data frame with two columns: `column_name` and `missing_values`, showing the count of `NA`s in each column of the input data frame.
- This function will require the `{dplyr}` package. Use `usethis::use_package("dplyr")` to declare this dependency in your `DESCRIPTION` file.

3. **Documentation:**

   - Use `roxygen2` comments to document your `summarize_df` function.
   - Include `@param`, `@return`, and `@export` tags.
   - Provide a working example in the `@examples` tag.
   - Since your function will use functions from `{dplyr}`, add the appropriate `@importFrom` tags (e.g., `@importFrom dplyr %>% summarise_all`).
   - Run `devtools::document()` to generate the documentation.

4. **Unit Tests:**

   - Set up testing with `usethis::use_testthat()`.
   - Create a test file with `usethis::use_test("summarize_df")`.
   - In the test file, write at least one test using `test_that()` that creates a sample data frame with missing values and checks if your function returns the correct counts.
   - Run `devtools::test()` to execute your tests.

5. **Quality Check:** Run `devtools::check()` to ensure your package is free of errors, warnings, and notes.

6. **GitHub Repository:** Push your completed R package to a new public repository on GitHub.

7. **Nix Expression:** Create a `default.nix` file in the root of your `datasummary` project. This file must:

    - Define a custom R environment.
    - Define your `datasummary` package by using `buildRPackage`. Fetch the source from your GitHub repository using `pkgs.fetchgit` (you will need the `rev` and `sha256`).
    - Make sure to list its R dependencies (like `{dplyr}` and its own dependencies) in `propagatedBuildInputs`.
    - Create a final shell with `pkgs.mkShell` that drops you into an R session where your `datasummary` package is installed and available.
    - **Hint:** Refer to the main course repository's `default.nix` file to see how R packages are defined with `buildRPackage` and included in the final R environment. Using `{rix}` is also an option.

To verify your work: * Drop into the Nix shell provided by your `default.nix`. * Start an R session. * Successfully run `library(datasummary)` and test your `summarize_df()` function on a data frame like `iris`.

7 Containerisation With Docker



7.1 Introduction

Up until now, we've built reproducible environments with Nix, written testable code with FP principles, orchestrated pipelines

with T, and made them robust with error handling and debugging. Now we turn to **distribution**: how do we share our work with others who may not have Nix installed?

Nix gives us fine-grained control over every package and dependency in our project, ensuring bit-for-bit reproducibility. However, when it comes to distributing a *data product*, another technology, Docker, is incredibly popular.

A common misconception is that Nix and Docker are alternatives, that you must choose one or the other. This is wrong. They are conceptually different tools that solve different problems. Nix is a **package manager and build system**: it answers "what software do I need and how do I build it reproducibly?" Docker is a **containerisation platform**: it answers "how do I package and run an application in isolation?" Understanding this distinction is key to using them together effectively.

While Nix manages dependencies for an application that runs on a host operating system, Docker takes a different approach: it packages an application *along with* a lightweight operating system and all its dependencies into a single, portable unit called a **container**. This container can then run on any machine that has Docker installed, regardless of its underlying OS. Being familiar with both tools makes you a more versatile data scientist: you can use Nix for precise, reproducible development environments and Docker for distributing your work to colleagues, servers, or CI pipelines that may not have Nix installed.

7.1.1 Spatial vs Temporal Reproducibility

To understand when and why to use Docker, it helps to distinguish between two types of reproducibility:

- **Spatial reproducibility**: The ability to execute an analysis identically across different machines *right now*. Docker excels here: a container runs the same way on your laptop, a colleague's workstation, and a cloud server.
- **Temporal reproducibility**: The ability to execute an analysis identically *over time*. Docker is weaker here. The imperative commands in a `Dockerfile` (like `apt-get update`) are non-deterministic. Rebuilding the same file at different times can yield different images, a temporal drift that is a well-documented weakness of container-based environments (Malka, Zacchiroli, and Zimmermann 2024).

This distinction is crucial. Docker's true reproducibility promise is that a specific, pre-built image will always launch an identical container. It does *not* promise that building the same `Dockerfile` twice will yield an identical image.

As empirical research has shown, achieving deterministic builds requires systems designed for this purpose, like Nix's functional model, where identical inputs always produce identical outputs. Studies rebuilding over 700,000 historical Nix packages from snapshots spanning several years found bit-for-bit reproducibility rates exceeding 90% (Malka, Zacchiroli, and Zimmermann 2025).

7.1.2 The Best of Both Worlds

This is why we combine Nix and Docker rather than choosing one or the other:

- **Nix** provides strong temporal reproducibility: the guarantee that an environment built today will be identical when rebuilt years later.

- **Docker** provides strong spatial reproducibility and universal distribution: the guarantee that a container runs the same everywhere Docker runs.

The pattern we'll use is simple: use Nix *inside* a Docker container, so that the container is both small and reproducible. Docker becomes a portable runtime for a Nix-managed environment, excellent for deployment to systems where Nix isn't installed. There are three ways to get Nix into the image:

1. **Install Nix inside the container** from a minimal base image. You add a few lines to the Dockerfile that install Nix, and everything after that is installed with Nix.
2. **Start from a base image that already has Nix.** You pull an existing image with Nix installed and use it as the foundation, skipping the installation step.
3. **Build the whole image with Nix.** You write the image definition in Nix itself, and Nix produces the container image for you.

This chapter covers all three, starting with the first two.

7.1.3 Interactive Development vs Distribution

This approach also clarifies *when* to use each tool:

- **Use Nix directly** for interactive development. It integrates seamlessly with your IDE and filesystem. No volume mounts, no port forwarding, no graphical application headaches.
- **Use Docker** for distributing finished data products. Package your T pipeline into an image that anyone can run with a single command.

If you've never heard of Docker before, this chapter will provide the basic knowledge required to get started. Let's start by watching this very short video[1] that introduces the core concepts.

The Rocker Project[2] provides a large collection of ready-to-use Docker images for R users.

7.2 Docker Essentials

7.2.1 Installing Docker

The first step is to install Docker. You'll find the instructions for Ubuntu here[3], for Windows here[4] (read the system requirements section as well!) and for macOS here[5] (make sure to choose the right version for the architecture of your Mac, if you have an M1 Mac use *Mac with Apple silicon*).

After installation, it might be a good idea to restart your computer, if the installation wizard does not invite you to do so. To check whether Docker was installed successfully, run the following command in a terminal:

```
docker run --rm hello-world
```

This should print the following message:

[1] https://www.youtube.com/watch?v=-X3AeROGmOw
[2] https://rocker-project.org/
[3] https://docs.docker.com/engine/install/ubuntu/#install-using-the-repository
[4] https://docs.docker.com/desktop/install/windows-install/
[5] https://docs.docker.com/desktop/install/mac-install/

```
Hello from Docker!
This message shows that your installation appears to
be working correctly.

To generate this message, Docker took the following
steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image
 from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from
 that image which runs the
    executable that produces the output you are
    currently reading.
 4. The Docker daemon streamed that output to the
 Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an
Ubuntu container with:
 $ docker run -it ubuntu bash

Share images, automate workflows, and more with a
free Docker ID:
 https://hub.docker.com/

For more examples and ideas, visit:
 https://docs.docker.com/get-started/
```

If you see this message, congratulations, you are ready to run Docker. If you see an error message about permissions, this means that something went wrong. If you're running Linux, make sure that your user is in the Docker group by running:

```
groups $USER
```

You should see your username and a list of groups that your user belongs to. If a group called `docker` is not listed, then you should add yourself to the group by following these steps[6].

7.2.2 The Rocker Project and Image Registries

When running a command like:

```
docker run --rm hello-world
```

what happens is that an image, in this case `hello-world`, gets pulled from a so-called *registry*. A registry is a storage and distribution system for Docker images. Think of it as a GitHub for Docker images, where you can push and pull images, much like you would with code repositories. The default public registry that Docker uses is called Docker Hub, but companies can also host their own private registries to store proprietary images.

Many open source projects build and distribute Docker images through Docker Hub, for example the Rocker Project.

The Rocker Project is instrumental for R users that want to use Docker. The project provides a large list of images that are ready to run with a single command. As an illustration, open a terminal and paste the following line:

```
docker run --rm -e PASSWORD=yourpassword -p
 ↪  8787:8787 rocker/rstudio
```

[6]https://docs.docker.com/engine/install/linux-postinstall/

Once this stops running, go to `http://localhost:8787/` and enter `rstudio` as the username and `yourpassword` as the password. You should log in to a RStudio instance: this is the web interface of RStudio that allows you to work with R from a server. In this case, the *server* is the Docker container running the image. Yes, you've just pulled a Docker image containing Ubuntu with a fully working installation of RStudio web!

Let's open a new script and run the following lines:

```
data(mtcars)
summary(mtcars)
```

You can now stop the container (by pressing `CTRL-C` in the terminal). Let's now rerun the container… you should realise that your script is gone! This is the first lesson: whatever you do inside a container will disappear once the container is stopped. This also means that if you install the R packages that you need while the container is running, you will need to reinstall them every time.

Thankfully, the Rocker Project provides a list of images with many packages already available. For example to run R with the `{tidyverse}` collection of packages already pre-installed, run:

```
docker run --rm -e PASSWORD=yourpassword -p
 ↪  8787:8787 rocker/tidyverse
```

7.2.3 Basic Docker Workflow

You already know about running containers using `docker run`. With the commands we ran before, your terminal will need to

stay open, or else, the container will stop. Starting now, we will run Docker commands in the background using the `-d` flag (*d* as in *detach*):

```
docker run --rm -d -e PASSWORD=yourpassword -p
 ↪  8787:8787 rocker/tidyverse
```

You can run several containers in the background simultaneously. List running containers with `docker ps`:

```
docker ps
CONTAINER ID   IMAGE              COMMAND   CREATED
 ↪  STATUS         PORTS                     NAMES
c956fbeeebcb   rocker/tidyverse   "/init"   3
 ↪  minutes ago   Up 3 minutes
 ↪  0.0.0.0:8787->8787/tcp   elastic_morse
```

Stop the container using its ID:

```
docker stop c956fbeeebcb
```

Let's discuss the other flags:

- `--rm`: Removes the container once it's stopped
- `-e`: Provides environment variables to the container (e.g., `PASSWORD`)
- `-p`: Sets the port mapping (host:container)
- `--name`: Gives the container a custom name

Run a container with a name:

```
docker run -d --name my_r --rm -e
 ↪  PASSWORD=yourpassword -p 8787:8787
 ↪  rocker/tidyverse
```

You can now interact with this container using its name:

```
docker exec -ti my_r bash
```

You are now inside a terminal session, inside the running container! This can be useful for debugging purposes.

Finally, let's solve the issue of our scripts disappearing. Create a folder somewhere on your computer, then run:

```
docker run -d --name my_r --rm -e
 ↪  PASSWORD=yourpassword -p 8787:8787 \
  -v
   ↪  /path/to/your/local/folder:/home/rstudio/scripts:rw
   ↪  rocker/tidyverse
```

You should now be able to save scripts inside the `scripts/` folder from RStudio and they will appear in the folder you created on your host machine.

7.3 Making Our Own Images

To create our own images, you can start from an image provided by an open source project like Rocker, or you can start from the base Ubuntu image. Since we are using Nix to set up the reproducible development environment, we can use `ubuntu:latest`. Our development environment will always be exactly the same, thanks to Nix.

7.3.1 A Minimal Dockerfile with Nix

```
FROM ubuntu:latest

RUN apt update -y
RUN apt install curl -y

# Install Nix via Determinate Systems installer
RUN curl --proto '=https' --tlsv1.2 -sSf -L
 ↪  https://install.determinate.systems/nix | sh -s
 ↪  -- install linux \
  --extra-conf "sandbox = false" \
  --init none \
  --no-confirm

# Add Nix to the path
ENV PATH="${PATH}:/nix/var/nix/profiles/default/bin"
ENV user=root

# Configure rstats-on-nix cache
RUN mkdir -p /root/.config/nix && \
    echo "substituters = https://cache.nixos.org
     ↪  https://rstats-on-nix.cachix.org" >
     ↪  /root/.config/nix/nix.conf && \
    echo "trusted-public-keys =
     ↪  cache.nixos.org-1:6NCHdD59X431o0gWypbMrAURkbJ16ZPMQF
     ↪  rstats-on-nix.cachix.org-1:vdiiVgocg6WeJrODIqdprZRUr
     ↪  >> /root/.config/nix/nix.conf

# Copy the T project's Nix files
COPY flake.nix flake.lock tproject.toml .

# Build the environment from the flake
```

```
RUN nix build --accept-flake-config .

# Enter the flake development shell when the
 ↪  container starts
CMD ["nix", "develop", "--command", "bash"]
```

Every `Dockerfile` starts with a `FROM` statement specifying the base image. Then, every command starts with `RUN`. We install and configure Nix, copy the T project's Nix files, and build the environment from the flake. Finally, when the container starts, `nix develop` drops you into the project's development shell (the `CMD` statement). This is the flake-native command: unlike the legacy `nix-shell`, which looks for a `shell.nix` or `default.nix` file, `nix develop` uses your `flake.nix`, consistent with the `nix build --accept-flake-config .` step above. Note that it expects an interactive terminal, so run the container with `docker run -it`.

7.3.2 Splitting Into a Reusable Base Image

This image actually does two things:

- a first step which consists in setting up Nix inside Docker;
- a second step which consists in setting up our project-specific Nix development environment.

Because the first step is generic, we can split this into two stages. First, create a new Dockerfile in a separate directory, for example `nix-base/`, with a new Git repository so that you can commit and push it. Later in this chapter we will publish this image, and in a continuous integration setup you could build and publish it automatically:

```
# nix-base/Dockerfile
FROM ubuntu:latest AS nix-base

RUN apt update -y && apt install -y curl

# Install Nix via Determinate Systems installer
RUN curl --proto '=https' --tlsv1.2 -sSf -L
 ↪  https://install.determinate.systems/nix | sh -s
 ↪  -- install linux \
  --extra-conf "sandbox = false" \
  --init none \
  --no-confirm

ENV PATH="/nix/var/nix/profiles/default/bin:${PATH}"
ENV user=root

# Configure Nix binary cache
RUN mkdir -p /root/.config/nix && \
    echo "substituters = https://cache.nixos.org
     ↪  https://rstats-on-nix.cachix.org" >
     ↪  /root/.config/nix/nix.conf && \
    echo "trusted-public-keys =
     ↪  cache.nixos.org-1:6NCHdD59X431o0gWypbMrAURkbJ16ZPMQF
     ↪  rstats-on-nix.cachix.org-1:vdiiVgocg6WeJrODIqdprZRUr
     ↪  >> /root/.config/nix/nix.conf
```

Build and tag this image:

```
docker build -t nix-base:latest .
```

Now, for any project, simply reuse it:

```
FROM nix-base:latest

COPY flake.nix flake.lock tproject.toml .

RUN nix build --accept-flake-config .

CMD ["nix", "develop", "--command", "bash"]
```

The issue with this approach is that you've created a dependency between the two Dockerfiles that you need to manage. I'd only recommend the second approach if you can push the first image to a registry, either a public one or a private one from your company. Later in this chapter we will publish the first image.

In a T project, the environment is declared in `tproject.toml` rather than generated by a script. For this example:

```
[r-dependencies]
packages = ["dplyr", "ggplot2"]

[py-dependencies]
version = "python313"
packages = ["polars", "great-tables"]
```

Build and run:

```
docker build -t my-project .
docker run -it --rm --name my-project-container
 ↪  my-project
```

This drops you in an interactive Nix shell running inside Docker!

7.3.3 Making T Available in the Image

So far, our images contain Nix and the packages we asked Nix to install. If your project uses T, you also need the `t` executable inside the container. T is distributed exclusively through Nix and is never installed as a system program, so the natural approach is to build it from the T flake and put `t` on the `PATH`. Add a step to the Dockerfile:

```
RUN nix build --accept-flake-config
 ↪  github:b-rodrigues/tlang \
    && ln -s "$(readlink -f result)/bin/t"
     ↪  /usr/local/bin/t
```

`nix build` compiles (or fetches from the rstats-on-nix cache) the T package and leaves a `result` symlink pointing at the built store path. The `ln` command creates a permanent symlink from `/usr/local/bin/t` to the `t` executable inside that store path. Because the symlink points at an absolute path in the Nix store, it survives across Docker layers, and `t` keeps working in the final image.

The `t` executable is a wrapper that already points at the R, Python, and Julia runtimes it needs. All of those runtimes live in the same Nix store, and Nix downloaded them when it built T, so the symlinked `t` runs without any extra setup.

For a fully reproducible build, pin the exact version of T that your project uses rather than the latest one on GitHub. A T project records its T version in `flake.lock`, so build that specific version:

```
RUN nix build --accept-flake-config
 ↪  github:b-rodrigues/tlang/v0.55.0 \
    && ln -s "$(readlink -f result)/bin/t"
     ↪  /usr/local/bin/t
```

Either way, `t` is available in the container without ever being installed as a system program.

7.4 Publishing Images on Docker Hub

If you want to share Docker images through Docker Hub, you first need to create a free account. A free account gives you unlimited public repositories.

List all images on your computer:

```
docker images
```

Log in to Docker Hub:

```
docker login
```

Tag the image with your username:

```
docker tag IMAGE_ID your_username/nix-base:latest
```

Push the image:

```
docker push your_username/nix-base:latest
```

This image can now be used as a stable base for other projects:

```
FROM your_username/nix-base:latest

RUN mkdir ...
```

7.4.1 Sharing Without Docker Hub

If you can't upload to Docker Hub, you can save the image to a file:

```
docker save nix-base | gzip > nix-base.tgz
```

Load it on another machine:

```
gzip -d nix-base.tgz
docker load < nix-base.tar
```

7.5 What If You Don't Use Nix?

Using Nix inside of Docker makes it very easy to set up an environment, but what if you can't use Nix for some reason? In this case, you would need to use other tools to install the right R or Python packages and it is likely going to be more difficult. The main issue you will face is missing development libraries.

For example, to install and use the R `{stringr}` package, you will need to first install `libicu-dev`:

```
FROM rocker/r-ver:4.5.1

RUN apt-get update && apt-get install -y \
    libglpk-dev \
    libxml2-dev \
    libcairo2-dev \
    libgit2-dev \
    libcurl4-openssl-dev \
    # ... many more
```

Another issue is that building the image is not a reproducible process, only running containers is. To mitigate this, use tagged images or better yet, a digest:

```
FROM
↪ rocker/r-ver@sha256:1dbe7a6718b7bd8630addc45a32731624fb7b7
```

This will always pull exactly the same layers. However, at some point, that version of Ubuntu will be outdated. Using Nix, you can stay on `ubuntu:latest`.

7.6 Building Docker Images Directly with Nix

So far, we've used Nix *inside* Docker containers. But there's an even more powerful approach: using Nix to build Docker images directly, bypassing `Dockerfile`s entirely.

Nix provides `dockerTools`, a set of functions that can create OCI-compliant container images. Because these images are built

through Nix's deterministic build system, they have stronger reproducibility guarantees than images built with `docker build`.

7.6.1 Why Skip the Dockerfile?

A `Dockerfile` is fundamentally imperative: it's a script of commands executed in order. Commands like `apt-get update` are non-deterministic by nature. Even with careful pinning, you're fighting the tool's design.

Nix's `dockerTools.buildImage` is declarative. You describe what should be in the image, and Nix figures out how to build it reproducibly. The resulting image is a pure function of its inputs.

7.6.2 A Basic Example

Here's a Nix expression that builds a Docker image containing R and some packages:

```
# docker-image.nix
let
  pkgs = import (fetchTarball

    ↪  "https://github.com/rstats-on-nix/nixpkgs/archive/202
  ) {};

  # Define the R environment
  myR = pkgs.rWrapper.override {
    packages = with pkgs.rPackages; [
      dplyr
```

```
      ggplot2
      quarto
    ];
  };

in pkgs.dockerTools.buildImage {
  name = "my-r-env";
  tag = "latest";

  copyToRoot = pkgs.buildEnv {
    name = "image-root";
    paths = [ myR pkgs.quarto pkgs.coreutils
     ↪  pkgs.bash ];
    pathsToLink = [ "/bin" ];
  };

  config = {
    Cmd = [ "${pkgs.bash}/bin/bash" ];
    WorkingDir = "/work";
  };
}
```

Build and load it:

```
# Build the image (outputs a .tar.gz file)
nix build -f docker-image.nix

# Load into Docker
docker load < result

# Run it
docker run -it --rm my-r-env:latest
```

7.6.3 Using T with dockerTools

The previous example manually defines the R environment in Nix. If you prefer to let T define your environment (as we've done throughout this book), you can build the Docker image from your T project's flake.

First, make sure your `tproject.toml` declares the packages you need:

```
[r-dependencies]
packages = ["dplyr", "ggplot2", "quarto"]
```

Then run `t update` to generate the `flake.nix` and `flake.lock` files. Now create a `docker-image.nix` that builds the image from the project's flake:

```
# docker-image.nix
let
  pkgs = import (fetchTarball

    ↪  "https://github.com/rstats-on-nix/nixpkgs/archive/202
  ) {};

  # The T project is a Nix flake. Load it and take
   ↪  the packages from its
  # development shell as the image's build inputs.
  project  = builtins.getFlake (toString
   ↪  ./flake.nix);
  devShell = project.devShell;

in pkgs.dockerTools.buildImage {
  name = "my-t-project";
```

```
  tag = "latest";

  copyToRoot = pkgs.buildEnv {
    name = "image-root";
    paths = devShell.buildInputs ++ [ pkgs.coreutils
     ↪  pkgs.bash ];
    pathsToLink = [ "/bin" ];
  };

  config = {
    Cmd = [ "${pkgs.bash}/bin/bash" ];
    WorkingDir = "/work";
  };
}
```

This approach lets you keep your familiar T workflow for defining environments while gaining the reproducibility benefits of `dockerTools` for image creation. Any changes to your `tproject.toml` → `flake.nix` will automatically propagate to the Docker image on the next build.

7.6.4 Benefits Over Dockerfiles

Aspect	Dockerfile	Nix dockerTools
Reproducibility	Imperative, non-deterministic	Declarative, deterministic
Layer caching	Based on command order	Based on content hashes
Image size	Often includes unnecessary packages	Minimal: only what you specify

Aspect	Dockerfile	Nix dockerTools
Composition	Copy-paste between files	Reuse Nix expressions

7.6.5 Layered Images for Efficiency

For faster CI builds, you can create layered images where the base environment is cached:

```
pkgs.dockerTools.buildLayeredImage {
  name = "my-analysis";
  tag = "latest";

  contents = [ myR pkgs.quarto ];

  config = {
    Cmd = [ "${myR}/bin/R" "--vanilla" ];
  };
}
```

`buildLayeredImage` creates separate layers for each package, so unchanged dependencies are cached between builds.

7.6.6 When to Use This Approach

- **CI/CD pipelines**: When you need reproducible image builds as part of automated workflows
- **Minimal images**: When image size matters and you want only what you need

- **Complex environments**: When the environment is already defined in Nix and you want to deploy it as a container

For simpler use cases, the "Nix inside Docker" approach from earlier sections may be more accessible.

7.7 Dockerising a T Pipeline

We can package our entire T project into a single Docker image. This image can then be run by anyone with Docker installed, regardless of their host operating system or whether they have Nix.

Assume your project directory has:

```
.
  data/
      mtcars.csv
  src/
      pipeline.t
      functions.R
      functions.py
  tproject.toml
  flake.nix
  flake.lock
```

7.7.1 Step 1: The Dockerfile

```
FROM nix-base:latest
# Or: FROM your-username/nix-base:latest

WORKDIR /app

# Copy all project files
COPY . .

# Build the pipeline during image build.
# `nix develop` enters the project's devShell, which
 ↪  puts `t` on the PATH.
RUN nix develop --accept-flake-config --command t
 ↪  run src/pipeline.t

# Export the pipeline artifacts when the container
 ↪  runs
CMD ["bash", "-c", "cp -r _pipeline /output"]
```

7.7.2 Step 2: The Pipeline

The pipeline lives in `src/pipeline.t`. It loads the data in T and produces the result as a node artifact:

```
-- src/pipeline.t
p = pipeline {
  data = read_csv("data/mtcars.csv")

  mtcars_head = rn(
    command      = <{ head(data, 10) }>,
    deserializer = ^ipc,
    serializer   = ^json
  )
```

```
}

build_pipeline(p, verbose = 1)
```

When T builds the pipeline, every node's artifact is written to the `_pipeline/` directory. The `mtcars_head` node's result is stored there as JSON, ready to be exported.

7.7.3 Step 3: Build and Run

Build:

```
docker build -t my-reproducible-pipeline .
```

Run to get results:

```
mkdir -p ./output

docker run --rm --name my_pipeline_run \
  -v "$(pwd)/output":/output \
  my-reproducible-pipeline
```

Check your local `output` directory. You'll find the pipeline's artifacts, including the `mtcars_head` result, containing the exact, reproducible output of your pipeline.

You have successfully packaged a complex, polyglot pipeline into a simple, portable Docker image. This workflow combines the best of both worlds: Nix's power for creating reproducible builds and Docker's universal standard for distributing and running applications.

7.7.4 Alternative: Using dockerTools for T

You can also build your T pipeline image purely with Nix, avoiding Dockerfiles entirely. This gives you fully deterministic image builds.

Create a `docker-pipeline.nix`:

```
# docker-pipeline.nix
let
  pkgs = import (fetchTarball

     ↪  "https://github.com/rstats-on-nix/nixpkgs/archive/202
  ) {};

  # The T project is a Nix flake. Load its
   ↪  development environment.
  project  = builtins.getFlake (toString
   ↪  ./flake.nix);
  devShell = project.devShell;

  # Build the pipeline as a derivation
  pipelineResult = pkgs.runCommand "pipeline-result"
   ↪  {
    buildInputs = devShell.buildInputs;
  } ''
    mkdir -p $out
    cd ${./. }
    t run src/pipeline.t
    # Copy results to output
    cp -r _pipeline $out/
  '';

in pkgs.dockerTools.buildImage {
```

```
  name = "my-t-pipeline";
  tag = "latest";

  copyToRoot = pkgs.buildEnv {
    name = "image-root";
    paths = [
      devShell.buildInputs
      pipelineResult
      pkgs.coreutils
      pkgs.bash
    ];
    pathsToLink = [ "/bin" "/" ];
  };

  config = {
    Cmd = [ "${pkgs.bash}/bin/bash" "-c" "cp -r
     ↪  /pipeline-result/* /output/" ];
    WorkingDir = "/work";
  };
}
```

Build and run:

```
nix build -f docker-pipeline.nix
docker load < result
docker run --rm -v "$(pwd)/output":/output
 ↪  my-t-pipeline:latest
```

This approach bakes the pipeline results directly into the image during the Nix build phase. The resulting image is minimal and contains only the outputs, not the full R/Python environment needed to produce them.

7.8 Summary

Docker and Nix complement each other:

- **Docker** provides a universal runtime and distribution mechanism
- **Nix** provides bit-for-bit reproducible environment management
- **Together** they enable truly reproducible, portable data products

Key Docker concepts:

- **Images** are templates; **containers** are running instances
- Containers are ephemeral; use **volumes** for persistence
- Use **registries** (Docker Hub) to share images
- The **Rocker Project** provides ready-made R images

For reproducibility:

- Use `ubuntu:latest` + Nix rather than pinning specific OS versions
- Build a reusable `nix-base` image to share across projects
- Get Nix into the image by installing it in the Dockerfile, starting from a base image that already has Nix, or building the whole image with Nix
- Build T from its flake and symlink `t` onto the `PATH`
- Package T pipelines for distribution

7.9 Further Reading

- Running Your R Script in Docker[7]

[7] https://www.r-bloggers.com/2019/02/running-your-r-script-in-docker/

- Docker & R Reproducibility[8]
- R Docker Tutorial[9]

[8] https://colinfay.me/docker-r-reproducibility/
[9] https://jsta.github.io/r-docker-tutorial/

8 Continuous Integration with GitHub Actions

8.1 Introduction

We are almost at the end of our journey. In the previous chapters, we built reproducible environments with Nix, organised our code into pure functions, made them robust with error handling and debugging, proved correctness with unit tests, managed our collaboration with Git, and bundled everything into shareable packages. We can now run our pipelines in a 100% reproducible way.

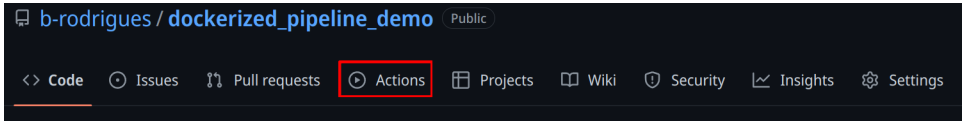
However, all of this still requires manual steps. And maybe that's not a problem; if your environment is set up and users only need to drop into a Nix shell and run the pipeline, that's already a huge improvement. But you should keep in mind that manual steps don't scale. Imagine you are part of a team that needs to quickly ship products to clients. Several people contribute to the product, and you might need to work on multiple projects in the same day. You and your teammates should be focusing on writing code, not on repetitive tasks like building images or running tests. Ideally, we would want to automate these steps. That is what we are going to learn in this chapter.

This chapter will introduce you to **Continuous Integration (CI)** with GitHub Actions. You will learn how to set up workflows that automatically run your tests when you push code, how to build Docker images and recover artifacts, and how to run your pipelines directly from GitHub's servers. Because we're using Git to trigger all the events and automate the whole pipeline, this approach is sometimes called GitOps.

You may have heard the term "CI/CD," where CD stands for Continuous Deployment or Continuous Delivery. We will focus on CI in this chapter. Continuous Deployment (automatically pushing results to a database, dashboard, or API) is highly specific to your organisation and infrastructure. What we cover here, however, gives you the foundation: once your pipeline runs reliably on CI, the "deployment" step is just one more workflow job pointing to wherever your results need to go.

8.2 Getting your repo ready for GitHub Actions

Obviously, you should use a project that is versioned on GitHub. Use the package we've developed previously. If you go on its GitHub page, you should see an "Actions" tab on top:



This will open a new view where you can select a lot of available, ready to use actions. "Actions" are premade scripts that execute some commands you might need: such as setting up R, Python, running tests, etc. Since we're using Nix, we don't really need to look for any actions to set up our environments. However, we

might want to use some pre-made actions to upload artifacts for instance.

To actually configure our repository to run actions, we need to edit a file in our project under the `.github/workflows` directory (create them if needed). In it, write a `yaml` file called `hello.yaml` and write the following in it:

```yaml
name: Hello world
on: [push]
jobs:
  say-hello:
    runs-on: ubuntu-latest
    steps:
      - run: echo "Hello from GitHub Actions!"
      - run: echo "This command is running from an
 ↪  Ubuntu VM each time you push."
```

Let's study this workflow definition line by line:

```
name: Hello world
```

Simply gives a name to the workflow.

```
on: [push]
```

When should this workflow be triggered? Here, whenever something gets pushed.

```
jobs:
```

What is the actual things that should happen? This defines a list of actions.

```
  say-hello:
```

This defines the `say-hello` job.

```
runs-on: ubuntu-latest
```

This job should run on an Ubuntu VM. You can also run jobs on Windows or macOS VMs, but this uses more compute minutes than a Linux VM (which doesn't matter for public projects; for private projects, the amount of compute minutes is limited).

```
steps:
```

What are the different steps of the job?

```
  - run: echo "Hello from GitHub Actions!"
```

First, run the command `echo "Hello from GitHub Actions!"`. This command runs inside the VM. Then, run this next command:

```
  - run: echo "This command is running from an
  Ubuntu VM each time you push."
```

If we take a look at the commit we just pushed, on GitHub, we see this yellow dot next to the commit name. This means that an action is running. We can then take a look at the output of the job, and see that our commands, defined with the `run` statements in the workflow file, succeeded and echoed what we asked them.

8.3 Nix and GitHub Actions

To set up Nix on GitHub Actions you can use several steps (create a new file called `run-tests.yaml`):

```
- name: Install Nix
  uses: cachix/install-nix-action@v31
  with:
    nix_path:
 ↪  nixpkgs=https://github.com/rstats-on-nix/nixpkgs/archive/

- name: Setup Cachix
  uses: cachix/cachix-action@v15
  with:
    name: rstats-on-nix
```

If your repository contains a `flake.nix` and `flake.lock` (as a T project does), the same environment you've been using locally can be used on GitHub Actions just as easily. The committed flake pins the exact T version and every dependency, so there is nothing to generate: `nix develop` reads it directly:

```
- name: Run in the project environment
  run: nix develop --command t --version
```

You can then use the shell to run whatever you need. For example, if you're developing a package, you could run unit tests on each push:

```
- name: devtools::test() via nix develop
  run: nix develop --command Rscript -e
 ↪  "devtools::test(stop_on_failure = TRUE)"
```

`stop_on_failure = TRUE` is needed to make the step fail if there's an error, otherwise, the step would run successfully, even with failing tests.

Of course, if you're developing a Python package, use `nix develop --command pytest` instead to run the tests.

I highly recommend you run tests when pull requests get opened:

```
on:
  push:
    branches: [ "main" ]
  pull_request:
    branches: [ "main" ]
```

This will ensure that if someone contributes to your project, you know immediately if what they did breaks tests or not. If it does, ask them to fix the code until tests pass.

8.4 Running a T Pipeline on GitHub Actions

Now that we know how to set up Nix on GitHub Actions, running a T pipeline is straightforward. Recall from Chapter 4 that T is distributed exclusively via Nix: you never install it in the traditional sense. A T project pins the exact version of T it uses in its committed `flake.nix` and `flake.lock` files, and `nix develop` drops you into a shell where that pinned `t` executable is available alongside all of your R, Python, and Julia runtimes.

That means "installing T" on CI is really just "installing Nix." Once Nix is present, the project's own flake takes care of the rest. Here is a complete workflow that runs our pipeline on every push and pull request to `main`:

```yaml
name: T Pipeline

on:
  push:
    branches: [ "main" ]
  pull_request:
    branches: [ "main" ]

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Install Nix
        uses: cachix/install-nix-action@v31
        with:
          extra_nix_config: |
            experimental-features = nix-command
 ↪  flakes
            substituters = https://cache.nixos.org
 ↪  https://rstats-on-nix.cachix.org
            trusted-public-keys =
 ↪  cache.nixos.org-1:6NCHdD59X431o0gWypbMrAURkbJ16ZPMQFGspcD
 ↪  rstats-on-nix.cachix.org-1:vdiiVgocg6WeJrODIqdprZRUrhi1Jz

      - name: Setup Cachix
        uses: cachix/cachix-action@v15
        with:
          name: rstats-on-nix

      - name: Run pipeline
        run: nix develop --command t run --failfast
 ↪  src/pipeline.t
```

The `extra_nix_config` block enables Nix flakes and points Nix at the `rstats-on-nix` binary cache, so R packages are downloaded pre-built rather than compiled from source. The `Setup Cachix` step registers that cache so the runner can pull from it.

The important line is the last one:

```
run: nix develop --command t run --failfast
src/pipeline.t
```

`nix develop` enters the project's environment using the committed `flake.lock`, which is where the T version is pinned. We then run `t run` on `src/pipeline.t`, exactly as we would locally. The `--failfast` flag makes the step, and therefore the whole workflow, fail as soon as any node errors, which is what you want on CI: a red build should mean something is actually broken.

A couple of notes:

- If you have changed `tproject.toml` (added a package, bumped a version), run `t update` first to regenerate `flake.nix` and `flake.lock`, then commit those files, the same one-time sync you do locally.
- Chapter 8 showed the `pipeline_to_ga()` function, which generates a workflow like this one automatically, including artifact caching. Writing it by hand, as here, just makes each moving part explicit.

8.4.1 Caching T pipeline outputs between runs

Nix already caches builds in its store, but a fresh CI runner starts with an empty store, so every node is rebuilt from scratch on the

first run. T has native functions for shipping a pipeline's cached artifacts between runs, so a second run can skip the work a first run already did.

`export_artifacts(p, archive_path)` bundles the cached Nix artifacts of every node in `p` into a single portable archive file. All nodes must already exist in the local store, so you call it right after a successful build:

```
build_pipeline(p)
export_artifacts(p, "artifacts.tar")
```

On the next run, `import_artifacts()` restores those artifacts into the local Nix store before you build, so unchanged nodes are pulled from the archive instead of rebuilt:

```
import_artifacts("artifacts.tar")
build_pipeline(p)
```

The two-argument form, `import_artifacts(p, "artifacts.tar")`, additionally verifies that the imported store paths match the pipeline's expected signatures, catching a stale or mismatched archive.

To see what an archive contains without touching the local store, use `inspect_artifacts()`. It loads the archive into a temporary store and returns a data frame with one row per node:

```
inspect_artifacts("artifacts.tar")
-- columns: node, store_path, hash, size_bytes,
 ↪  references
```

On GitHub Actions the pattern is to export the archive after a successful build and upload it as a workflow artifact, then download and import it at the start of the next run. Combined with the `rstats-on-nix` binary cache above, this keeps even a cold runner from recompiling anything that has not changed.

8.5 Running a dockerized workflow

This next example can be found in this repository[1]. This example doesn't use Nix or T, but the point here is to show how a Docker container can be executed on GitHub Actions, and artifacts can be recovered. The process is always the same, regardless of what is inside the Docker image. If you want to follow along, fork this repository.

This is what our workflow file looks like:

```
name: Reproducible pipeline

on:
  push:
    branches: [ "main" ]
  pull_request:
    branches: [ "main" ]

jobs:

  build:

    runs-on: ubuntu-latest
```

[1] https://github.com/b-rodrigues/dockerized_pipeline_demo

```
    steps:
    - uses: actions/checkout@v5
    - name: Build the Docker image
      run: docker build -t my-image-name .
    - name: Docker Run Action
      run: docker run --rm --name
↪   my_pipeline_container -v
↪   /github/workspace/fig/:/home/graphs/:rw
↪   my-image-name
    - uses: actions/upload-artifact@v4
      with:
        name: my-figures
        path: /github/workspace/fig/
```

For now, let's focus on the `run` statements, because these should be familiar:

```
run: docker build -t my-image-name .
```

and:

```
run: docker run --rm --name my_pipeline_container -v
/github/workspace/fig/:/home/graphs/:rw
my-image-name
```

The only new thing here, is that the path has been changed to `/github/workspace/`. This is the home directory of your repository, so to speak. Now there's the `uses` keyword that's new:

```
uses: actions/checkout@v5
```

This action checks out your repository inside the VM, so the files in the repo are available inside the VM. Then, there's this action here:

```
- uses: actions/upload-artifact@v4
  with:
    name: my-figures
    path: /github/workspace/fig/
```

This action takes what's inside `/github/workspace/fig/` (which will be the output of our pipeline) and makes the contents available as so-called "artifacts". Artifacts are the outputs of your workflow. In our case, as stated, the output of the pipeline. So let's run this by pushing a change, and let's take a look at these artifacts!

After the action is done running, you will be able to download a zip file containing the plots. It is thus possible to rerun our workflow in the cloud. This has the advantage that we can now focus on simply changing the code, and not have to bother with boring manual steps. For example, let's change this target in the `_targets.R` file:

```
tar_target(
  commune_data,
  clean_unemp(
    unemp_data,
    place_name_of_interest = c(
      "Luxembourg", "Dippach",
      "Wiltz", "Esch/Alzette",
      "Mersch", "Dudelange"),
    col_of_interest = active_population)
)
```

I've added "Dudelange" to the list of communes to plot. Pushing this change to GitHub triggers the action we've defined before. The plots (artifacts) get refreshed, and we can download

them. Take a look and see that Dudelange was added in the `communes.png` plot!

It is also possible to "deploy" the plots directly to another branch, and do much, much more. I just wanted to give you a little taste of GitHub Actions (and more generally GitOps). The possibilities are virtually limitless, and I still can't get over the fact that GitHub Actions is free for public repositories.

8.6 Building a Docker image and pushing it to a registry

It is also possible to build a Docker image and have it made available on an image registry. You can see how this works on this repository[2]. These images can then be used as a base for other reproducible pipelines, as in this repository[3]. Why do this? Well because of "separation of concerns". You could have a repository which builds an image containing your development environment: this could be an image with a specific version of R and R packages built with Nix. And then have as many repositories as projects that run pipelines using that development environment image as a basis. Simply add the project-specific packages that you need for each project.

[2]https://github.com/b-rodrigues/ga_demo
[3]https://github.com/b-rodrigues/ga_demo_rap/tree/main

8.7 A Complete T Pipeline Workflow on GitHub Actions

The previous section showed the essential workflow in a few lines. Here is a complete, annotated version that also inspects the built pipeline and reads a node's result. Because Nix handles all dependencies reproducibly, you don't need Docker as an intermediary. The tlang[4] repository contains several complete examples; here we will walk through the key steps.

The workflow triggers on pushes and pull requests to `main`:

```
on:
  pull_request:
    branches: [main, master]
  push:
    branches: [main, master]
```

After checking out the repository and installing Nix (with Cachix for faster builds), there is no environment-generation step. A T project commits its `flake.nix` and `flake.lock`, which pin the exact T version and every dependency, and the pipeline itself is the committed `src/pipeline.t` file. `nix develop` reads the flake directly, so nothing needs to be generated.

The core step builds the pipeline exactly as you would locally:

```
- name: Build pipeline
  run: nix develop --command t run src/pipeline.t
```

[4] https://github.com/b-rodrigues/tlang

You can validate the pipeline's structure with `t check`, which inspects the pipeline without running any Nix builds:

```
- name: Check pipeline structure
  run: nix develop --command t check src/pipeline.t
```

Finally, read a node's result to confirm the pipeline produced what you expect:

```
- name: Show result
  run: nix develop --command t run --expr
  ↪  'read_node("confusion_matrix")'
```

8.7.1 Caching Pipeline Outputs Between Runs

While Nix caches derivations, CI runners are ephemeral: each run starts fresh. To avoid rebuilding the entire pipeline every time, T provides `export_artifacts()` and `import_artifacts()` to persist outputs between runs.

Before building, check if cached outputs exist and import them:

```
- name: Import cached outputs if available
  run: |
    if [ -f
  ↪  "../outputs/my_pipeline/pipeline_outputs.tar" ];
  ↪  then
      nix develop --command t import_artifacts
  ↪  src/pipeline.t
  ↪  ../outputs/my_pipeline/pipeline_outputs.tar
    else
```

```
      echo "No cached outputs found, will build from
 ↪  scratch"
    fi
```

After building, export the outputs so they can be reused:

```
- name: Export outputs to avoid rebuild
  run: |
    mkdir -p ../outputs/my_pipeline
    nix develop --command t export_artifacts
 ↪  src/pipeline.t
 ↪  ../outputs/my_pipeline/pipeline_outputs.tar
```

Finally, commit the cached outputs back to the repository:

```
- name: Push cached outputs
  run: |
    cd ..
    git config --global user.name "GitHub Actions"
    git config --global user.email
 ↪  "actions@github.com"
    git pull --rebase --autostash origin main
    git add outputs/my_pipeline/pipeline_outputs.tar
    if git diff --cached --quiet; then
      echo "No changes to commit."
    else
      git commit -m "Update cached pipeline outputs"
      git push origin main
    fi
```

This pattern ensures that only changed nodes are rebuilt on subsequent runs.

8.7.2 The Easy Way: `pipeline_to_ga()`

If the above seems like a lot of boilerplate, T provides a helper function that generates a complete GitHub Actions workflow for you:

```
pipeline_to_ga(p, file =
 ↪  ".github/workflows/pipeline.yml")
```

This creates a `.github/workflows/pipeline.yml` file that handles everything: installing Nix, setting up Cachix, building the pipeline, and importing and exporting artifacts. It stores the cached Nix store in a dedicated `t-runs` branch, keeping your main branch clean while persisting the cached outputs between runs.

For most projects, running `pipeline_to_ga()` once and committing the generated workflow file is all you need to get your pipeline running on CI.

8.8 GitHub Actions without Nix

If you're not using Nix, you'll have to set up GitHub Actions manually. Suppose you have a package project and want to run unit tests on each push. See for example the `{myPackage}` package, in particular this file[5]. This action runs on each push and pull request on Windows, Ubuntu and macOS:

[5] https://github.com/b-rodrigues/myPackage/blob/main/.github/workflows/rcmdcheck.yaml

```
on:
  push:
    branches: [ "main" ]
  pull_request:
    branches: [ "main" ]

jobs:
  rcmdcheck:
    runs-on: ${{ matrix.os }}
    strategy:
      matrix:
        os: [ubuntu-latest, windows-latest,
        macos-latest]
```

Several steps are executed, all using pre-defined actions from the `r-lib` project:

```
    steps:
    - uses: actions/checkout@v4
    - uses: r-lib/actions/setup-r@v2
    - uses: r-lib/actions/setup-r-dependencies@v2
      with:
        extra-packages: any::rcmdcheck
        needs: check
    - uses: r-lib/actions/check-r-package@v2
```

An action such as `r-lib/actions/setup-r@v2` will install R on any of the supported operating systems without requiring any configuration from you. If you didn't use such an action, you would need to define three separate actions: one that would be executed on Windows, on Ubuntu and on macOS. Each of these operating-specific actions would install R in their operating-specific way.

Check out the workflow results to see how the package could be improved here[6].

Here again, using Nix simplifies this process immensely. Look at this workflow file from T's repository here[7]. Setting up the environment is much easier, as is running the actual test suite.

8.9 Advanced patterns

Now that you understand the basics, let's look at some more advanced patterns that will make your CI workflows more efficient and informative.

8.9.1 Caching with Cachix

Building Nix environments from scratch on every CI run can be slow. Cachix solves this by providing a binary cache for your Nix derivations. Once you build something, subsequent runs can download the pre-built binaries instead of rebuilding from source.

To use Cachix, you first need to create a free account at cachix.org[8] and create a cache. Then, generate an auth token and add it as a secret in your GitHub repository settings (under Settings → Secrets and variables → Actions). Call it something like `CACHIX_AUTH`.

Here is a workflow that builds your development environment and pushes the results to your Cachix cache:

[6] https://github.com/b-rodrigues/myPackage/actions/runs/12348361696
[7] https://github.com/b-rodrigues/tlang/blob/main/.github/workflows/unit-tests.yaml
[8] https://cachix.org

```yaml
name: Update Cachix cache

on:
  push:
    branches: [main]

jobs:
  build-and-cache:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Install Nix
        uses:
 ↪  DeterminateSystems/nix-installer-action@main

      - uses: cachix/cachix-action@v15
        with:
          name: your-cache-name
          authToken: '${{ secrets.CACHIX_AUTH }}'

       - name: Build and push to cache
         run: |
           nix build .#devShell
           nix-store -qR --include-outputs $(nix
 ↪  eval --raw .#devShell) | cachix push
 ↪  your-cache-name
```

The key line here is the `nix-store` command at the end. It queries all the dependencies of your build and pushes them to Cachix. The next time you or anyone else runs this workflow, the `cachix/cachix-action` will automatically pull from your cache, dramatically speeding up the build.

If you want to build on both Linux and macOS (since Nix binaries are platform-specific), you can use a matrix:

```
jobs:
  build-and-cache:
    runs-on: ${{ matrix.os }}
    strategy:
      matrix:
        os: [ubuntu-latest, macos-latest]
```

8.9.2 Storing outputs in orphan branches

When running a pipeline on CI, you often want to keep the outputs (plots, data, reports) without committing them to your main branch. A clean solution is to store them in an orphan branch. An orphan branch has no commit history and is completely separate from your main code.

Here is the pattern:

```
- name: Check if outputs branch exists
  id: branch-exists
  run: git ls-remote --exit-code --heads origin
  ↪  pipeline-outputs
  continue-on-error: true

- name: Create orphan branch if needed
  if: steps.branch-exists.outcome != 'success'
  run: |
    git checkout --orphan pipeline-outputs
    git rm -rf .
    echo "Pipeline outputs" > README.md
```

```
    git add README.md
    git commit -m "Initial commit"
    git push origin pipeline-outputs
    git checkout -

- name: Push outputs to branch
  run: |
    git config --local user.name "GitHub Actions"
    git config --local user.email
 ↪  "actions@github.com"
    git fetch origin pipeline-outputs
    git worktree add ./outputs pipeline-outputs
    cp -r _outputs/* ./outputs/
    cd outputs
    git add .
    git commit -m "Update outputs" || echo "No
 ↪  changes"
    git push origin pipeline-outputs
```

This pattern first checks if the branch exists using `git ls-remote`. If not, it creates an orphan branch. Then it uses `git worktree` to work with both branches simultaneously, copies the outputs, and pushes them. T uses this pattern to store pipeline outputs between runs.

8.9.3 Creating workflow summaries

GitHub Actions has a built-in feature for creating rich summaries that appear directly on the workflow run page. You write Markdown to a special file path stored in the `GITHUB_STEP_SUMMARY` environment variable.

```
- name: Create summary
  run: |
    echo "## Pipeline Results  " >>
 ↪  $GITHUB_STEP_SUMMARY
    echo "" >> $GITHUB_STEP_SUMMARY
    echo "| Metric | Value |" >>
 ↪  $GITHUB_STEP_SUMMARY
    echo "|--------|-------|" >>
 ↪  $GITHUB_STEP_SUMMARY
    echo "| Tests passed | 42 |" >>
 ↪  $GITHUB_STEP_SUMMARY
    echo "| Coverage | 87% |" >>
 ↪  $GITHUB_STEP_SUMMARY
```

You can also generate the summary dynamically from your R or Python code:

```
- name: Generate summary from R
  run: |
    nix develop --command Rscript -e '
      results <- readRDS(\"results.rds\")
      cat(\"## Analysis Complete\n\n\", file =
 ↪  Sys.getenv(\"GITHUB_STEP_SUMMARY\"), append =
 ↪  TRUE)
      cat(paste(\"Processed\", nrow(results),
 ↪  \"observations\n\"), file =
 ↪  Sys.getenv(\"GITHUB_STEP_SUMMARY\"), append =
 ↪  TRUE)
    '"
```

This is particularly useful for:

- Showing test results at a glance

- Displaying key metrics from your analysis
- Providing download links to artifacts
- Reporting any warnings or issues

The summary appears right on the Actions tab, making it easy for collaborators to see what happened without digging through logs.

8.10 Conclusion

This chapter introduced Continuous Integration with GitHub Actions, the final piece of our reproducible workflow.

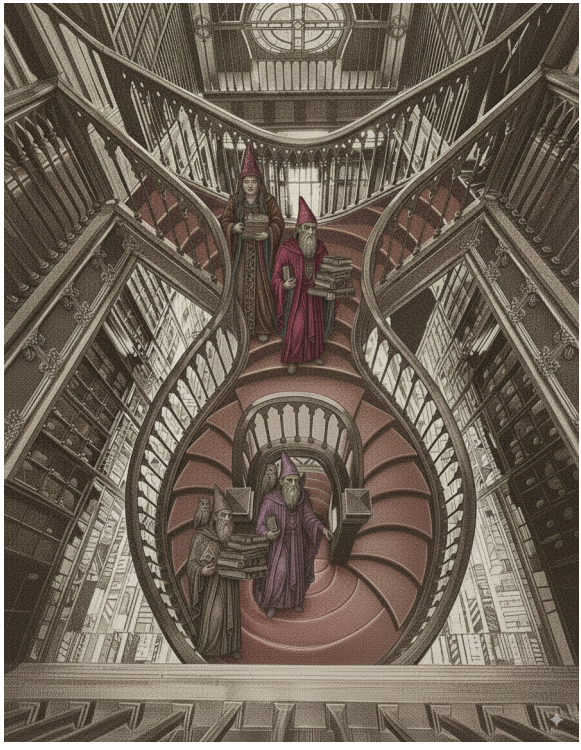
Key takeaways:

- **Automation removes manual steps**: Every push triggers tests, builds, and deployments without human intervention
- **Nix simplifies CI setup**: The same `flake.nix` you use locally works on GitHub Actions, eliminating "works on my machine" problems
- **Cachix speeds up builds**: By caching Nix derivations, subsequent runs avoid rebuilding unchanged dependencies
- **`pipeline_to_ga()` handles the boilerplate**: One function call generates a complete workflow for running T pipelines on CI
- **Artifact caching persists outputs**: Using `export_artifacts()` and an orphan branch, pipeline outputs survive between ephemeral CI runs

With continuous integration in place, your reproducible analytical pipeline is truly automated. Push your code, and GitHub takes care of the rest: running tests, building your environment,

executing your pipeline, and storing the results. This frees you to focus on what matters: the analysis itself.

9 Conclusion: Adopting Reproducibility in the Real World



9.1 Introduction

If you have made it this far, you now have a powerful toolkit at your disposal. You understand how Nix provides truly reproducible environments, how functional programming leads to testable and predictable code, how packages bundle your work for sharing, and how CI/CD automates the boring parts. But knowing these tools is only half the battle. The harder question is: how do you actually start using them?

If you work alone, the answer is simple: just start. Pick one project and commit to doing it the "right" way. You will make mistakes, you will get frustrated, and you will learn. But if you work in a team, the challenge is different. You cannot simply mandate that everyone learns Nix by next Tuesday. Change management is a skill in itself, and this final chapter offers some practical strategies for introducing reproducible practices to yourself, your team, and your organisation.

The philosophy here is simple: show, don't tell. Nobody was ever convinced to adopt a new tool by a slideshow. They are convinced when they see it solve a real problem.

9.2 Start with yourself

Before you try to convince anyone else, you need to become proficient yourself. Pick a small, low-stakes project and use it as your learning ground. A perfect candidate is a personal side project or an internal analysis that nobody else depends on.

Your goal in this phase is to build muscle memory. You want to reach the point where writing a `pipeline.t` feels as natural as opening RStudio. You want to be able to troubleshoot common

Nix errors without panic. You want to have a working CI pipeline that you understand inside and out.

This investment pays dividends later. When a colleague asks "Why is this so complicated?", you will have a ready answer because you asked yourself the same question two months ago.

9.3 The two-minute demo

Once you are comfortable, look for opportunities to demonstrate value. The best opportunities are problems that everyone recognises but nobody has solved.

"It works on my machine" is the classic. The next time a colleague struggles to run your code, don't just send them a fixed script. Send them the T project and a one-liner: `nix develop --command t run`. When the whole pipeline runs on the first try, you will have their attention.

Another opportunity is the dreaded "update everything and pray" scenario. When a project breaks after a package update, you can show how Nix lets you pin exact versions. "This analysis will run the same way in five years" is a powerful statement.

The key is to solve real problems, not hypothetical ones. Do not lecture your colleagues about reproducibility in the abstract. Show them how it saved you two hours of debugging on Tuesday.

9.4 Make it easy for others

If you want your team to adopt these practices, you need to lower the barrier to entry as much as possible. This means:

1. Write excellent documentation. Not for yourself, but for the colleague who has never heard of Nix. Assume they will spend five minutes reading before giving up.
2. Provide templates. Create a repository with a working T project: a `pipeline.t`, a `tproject.toml`, a CI workflow, and a simple example analysis. When someone starts a new project, they can clone this template and be productive immediately.
3. Automate the setup. If your organisation allows it, provide a script that installs Nix with a single command. The fewer steps, the fewer places where someone can get stuck.
4. Be available. When someone tries your approach and hits a wall, they need help within minutes, not days. If you are not responsive, they will abandon the effort and go back to their old ways.

9.5 Pick your battles

You cannot change everything at once. Be strategic about where you focus your energy.

Some projects are not worth the effort to convert. If an analysis was run once three years ago and will never be touched again, leave it alone. Focus on new projects and on projects that are actively maintained.

Some colleagues will never be convinced. That is fine. You do not need everyone on board. You need a few early adopters who will become advocates themselves. Focus on the curious ones, the ones who ask "how did you do that?" when they see your workflow.

Some organisations have constraints you cannot change. Maybe you cannot install Nix on the production server. Maybe you are stuck with Windows machines. Work within these constraints rather than fighting them. Docker can bridge many gaps. WSL2 makes Nix possible on Windows. Find the path of least resistance.

9.6 The gradual adoption path

Here is a realistic roadmap for introducing reproducibility to a team:

Month 1-2: Personal mastery. Use Nix and the tools from this book on your own projects. Get comfortable. Build your own templates and workflows.

Month 3-4: Show and tell. Start sharing your work. When you give a code review, mention that the project runs in a Nix shell. When someone asks how to add a package, show them that it is a line in `tproject.toml` followed by `t update`. Let people discover the value organically.

Month 5-6: Pilot project. Propose using the full stack (Nix, tests, CI) for one team project. Pick something visible but not critical. Document everything. Make it a success.

Month 7-12: Expand. Use the pilot project as a template for new work. Gradually, "the new way" becomes "the way we do

things here." Update onboarding documentation to include Nix setup.

This is slow. It takes patience. But lasting change always does.

9.7 Common objections and how to address them

"This is too complicated."

It is more complicated than doing nothing. But it is simpler than debugging environment issues at 3am before a deadline. Complexity is a trade-off. You are trading upfront learning for long-term reliability. Acknowledge the learning curve, but emphasise that you are there to help.

"We don't have time for this."

You do not have time not to do this. Every hour spent on environment issues is an hour not spent on analysis. Every "it works on my machine" is a risk. Frame reproducibility as a time investment, not a time cost.

"My code works fine without it."

It works today. Will it work in six months? Will it work when you hand it to a colleague? Will it work when the package maintainer pushes a breaking change? Reproducibility is insurance. You hope you never need it, but you are grateful when you do.

"I'll learn it later."

Later never comes. Start with one small project. Commit to using one new tool. You can learn incrementally. You do not need to master everything before you begin.

9.8 LLMs as adoption accelerators

Throughout this book, we have seen how LLMs can assist with writing tests, generating documentation, and even drafting functions. But there is a bigger point to make here: T was designed with LLMs in mind, and that design choice changes how you adopt everything else in this book.

The old objection to tools like Nix was that the learning curve was too steep. You had to understand a new language, a new paradigm, and a lot of unfamiliar syntax. T softens that objection in two ways. First, its syntax is simple enough that a human can read and write it comfortably, without a manual. Second, and more importantly, it was designed from the start to be written by LLMs. The nodes, the dependencies, the data flow: all of it maps cleanly onto the kind of structured, declarative description that language models are good at producing.

So who writes your pipeline? In practice, both. You can open a `pipeline.t` file and understand it. You can also hand a problem to an LLM and get a working pipeline back. The same file can be edited by a human on Monday and regenerated by a model on Tuesday, and both versions stay legible to both of you. That is a rare property, and it is what makes T a good fit for AI-assisted work.

But the language is only half the story. What actually determines whether an LLM contributes meaningfully is the context you give it, and the harness that surrounds it. Every T project ships with an `AGENTS.md` file and a `T-LANGUAGE-REFERENCE.md`. The `AGENTS.md` is where you tell the model how your project works: where the data lives, what the conventions are, what "done" looks like. The language reference gives it the precise syntax it needs. Together they turn a general-purpose model into something that

knows your project. `t init` generates both for you, which is why a brand-new project is already LLM-ready.

This is where the real investment is, and it is not in prompting. It is in thinking. Take the time to understand your project well enough to explain it. A model given a rich, accurate description of your data and your goals will produce something you can build on. A model given a vague one-liner will produce something plausible but wrong. The quality of the context is the quality of the result.

And there is a lot you can do, as a human, to make the model's life easier. You do not have to ask the LLM to do everything. If downloading and cleaning the raw data into a tidy, ready-to-use format is something you can do once by hand, do it, and store the result in the project. Now the model starts from clean input instead of spending its effort, and your review time, writing a script to scrape and wrangle data you could have prepared yourself. The same goes for pinning a tricky dependency, sketching the first version of the pipeline's nodes, or writing the hard function yourself. You are not offloading your thinking to the model; you are clearing the path so that when it does think, it thinks about the right things.

This concept of "clearing the path" is the foundation of modern **AI harness engineering**. For an AI assistant or autonomous agent to be genuinely efficient, you cannot simply drop it into an empty directory with an open-ended prompt. You must provide a structured harness built upon three essential pillars:

1. **Context:** The `AGENTS.md` file, schema definitions, and language references that explain the domain, conventions, and what "done" looks like.

2. **Deterministic Constraints:** The hermetic Nix environment and isolated, pure T nodes that prevent the model from drifting into dependency hell or mutating global state.
3. **Test Oracles and Evaluation Benchmarks:** The golden datasets, reference implementations, and invariant assertions that serve as objective arbiters of correctness.

Preparing these oracles upfront is arguably the highest-leverage work a data scientist can do. If you invest twenty minutes to prepare a small, verified golden dataset and write a handful of property-based tests, you can unleash an AI agent to draft the pipeline nodes, optimize data wrangling, or port algorithms across languages. The agent executes against the oracle, receives machine-readable diagnostics via `t test --json`, and self-corrects until it passes. You never have to wonder whether the AI hallucinated a subtle statistical error or corrupted an edge case: the oracle guarantees the result, allowing the agent to work at full throttle with complete safety.

The human's job, then, shifts from writing every line to reviewing and directing. The bottleneck is no longer "I need to learn everything"; it is "I need to understand enough to verify the output." That is a much smaller ask, and it is why the combination works. T provides the structure and the reproducibility. The LLM provides the speed and the accessibility. And the context you invest provides the accuracy. Together, they make best practices achievable for teams that would never have adopted them otherwise.

9.9 A note on literate programming

This book has focused on the plumbing of reproducibility: environments, packages, pipelines, and automation. But there

is another dimension we have not covered in depth: literate programming.

Literate programming is the practice of combining code, prose, and outputs into a single document. Tools like Quarto, R Markdown, and Jupyter notebooks (Kluyver et al. 2016) make this possible. The idea is that the analysis and its documentation are the same thing. You cannot have one without the other.

This book itself was written with Quarto. Every code example runs. Every output is generated fresh on each build. This is literate programming in action.

If you are interested in this approach, there are excellent resources available. The Quarto documentation at quarto.org[1] is a great starting point. For R Markdown, "R Markdown: The Definitive Guide" by Xie, Allaire, and Grolemund is comprehensive. The principles you have learned in this book apply directly: pin your environment with Nix, run your document rendering in CI, and your reports become just as reproducible as your analyses.

9.10 Final thoughts

Reproducibility is not a destination. It is a practice. You will never have a perfectly reproducible workflow because requirements change, tools evolve, and you learn new things. The goal is not perfection but continuous improvement.

Start where you are. Use what you have. Do what you can.

If this book has given you one new tool or one new idea, it has done its job. If it has changed how you think about your work, even better. And if you go on to share these practices with your

[1] https://quarto.org

colleagues and build a culture of reproducibility in your team, then you have not only learned but taught. That is the highest compliment.

Good luck. Stay curious. Build things that last.

Dolstra, Eelco. 2006. "The Purely Functional Software Deployment Model." PhD thesis, Utrecht University. https://edolstra.github.io/pubs/phd-thesis.pdf.

Kluyver, Thomas, Benjamin Ragan-Kelley, Fernando Pérez, Brian E. Granger, Matthias Bussonnier, Jonathan Frederic, Kyle Kelley, et al. 2016. "Jupyter Notebooks – a Publishing Format for Reproducible Computational Workflows." In *Positioning and Power in Academic Publishing: Players, Agents and Agendas*, edited by Fernando Loizides and Birgit Schmidt, 87–90. IOS Press. https://doi.org/10.3233/978-1-61499-649-1-87.

Malka, Julien, Stefano Zacchiroli, and Théo Zimmermann. 2024. "Reproducibility of Build Environments Through Space and Time." In *Proceedings of the 2024 ACM/IEEE 44th International Conference on Software Engineering: New Ideas and Emerging Results*, 97–101. ICSE-NIER'24. ACM. https://doi.org/10.1145/3639476.3639767.

———. 2025. "Does Functional Package Management Enable Reproducible Builds at Scale? Yes." In *22nd International Conference on Mining Software Repositories*. Ottawa, Canada. https://hal.science/hal-04913007.

Peng, Roger D. 2011. "Reproducible Research in Computational Science." *Science* 334 (6060): 1226–27.